NORTH PHARMACY · QUALITY ASSURANCE

API Documentation & Test Specification

North Pharmacy e-commerce platform

A complete reference to the 73 HTTP endpoints behind the North Pharmacy customer storefront and administration console, together with the test strategy, the API test catalogue and the automation that executes it. Every endpoint, payload, status code and limit in this document was observed against the live test environment — nothing here is inferred from design notes.

DOCUMENT	NP-API-TS-001
VERSION	1.0
STATUS	Issued for review
DATE	25 October 2026
ENVIRONMENT	https://202.4.101.118:1165
COMPANION TO	NP-QA-TC-001 v1.3 — UI test case specification (434 cases)
PREPARED BY	QA Automation — devteam@emslbd.com

Contents

1 Introduction

Purpose · Scope · Audience · Related documents · How this was produced

2 System under test

Architecture · Environment · Accounts · Reference data

3 API conventions

Transport · Authentication · The CSRF guard header · Error model · Status codes · Pagination · Rate limiting · Security headers

4 Endpoint reference — storefront

Store · Catalogue · Authentication · Cart · Wishlist · Account · Prescriptions · Checkout · Orders · Newsletter

5 Endpoint reference — administration

Dashboard · Orders · Prescriptions · Inventory · Customers · Manufacturers · Content · Settings · Reports

6 Test strategy

Objectives · Levels · Risk ranking · Environment and data · Entry and exit criteria · Defect handling

7 Test case catalogue

Cases across 19 modules, with priority and expected result

8 Security and authorisation tests

Authentication · Object-level authorisation · Injection · Upload abuse · Headers

9 Automation

Playwright structure · Fixtures · Worked examples · Running · CI

10 Findings from this survey

Eight verified observations, with impact and recommendation

11 Traceability and coverage

12 Appendices

Enumerations · Sample payloads · Glossary · Revision history

1 Introduction

1.1 Purpose

This document is the single reference for testing the North Pharmacy HTTP API. It serves three audiences at once: it **documents** what the API is, it **specifies** what must be tested about it, and it **records** what was observed when the API was surveyed on 25 October 2026.

The existing specification *NP-QA-TC-001 v1.3* covers the product through the browser: 434 cases driving pages, forms and screens. That approach is necessary but slow and indirect. A rule such as "maximum 200 units per product" is enforced by the server, and the cheapest, most reliable way to prove it is one HTTP request — not a browser, a cart page and a quantity stepper. This document covers that layer.

1.2 Scope

IN SCOPE	OUT OF SCOPE
All 73 endpoints under `/api/*`, customer and administration	The React single-page application and its rendering
Request and response contracts, including field presence and types	Database schema and stored procedures
Status codes, the RFC 7807 error model and validation messages	The point-of-sale system that feeds `soldAtPos`
Authentication, session handling and the CSRF guard	SMS gateway and e-mail delivery (not wired in this build)
Object-level authorisation between customers, and customer versus administrator	Payment gateway integration (only Cash on Delivery is configured)
Business rules the server owns: delivery charge, free-delivery threshold, quantity caps, prescription gating	Infrastructure hardening, TLS certificate issuance, WAF rules
Rate limiting on authentication endpoints	Sustained load and capacity — see the companion load test plan, NP-LT-001

1.3 Audience

- **QA engineers** — sections 3, 6, 7, 8 and 9 are the working material.
- **Backend developers** — sections 4, 5 and 10 describe the contract as it behaves today, which in several places differs from what the UI assumes.
- **Release and product management** — sections 6.5 and 10 carry the gating criteria and the open issues.

1.4 Related documents

REFERENCE	TITLE	RELATIONSHIP
NP-QA-TC-001 v1.3	North Pharmacy test case specification (434 UI cases)	Parent document; section 11 traces to it
NP-LT-001	Load and performance test plan	Companion; reuses this endpoint reference
DEFECTS.md	Defects found during UI automation	Section 10 extends it with API-layer findings

REFERENCE	TITLE	RELATIONSHIP
README.md	Playwright suite documentation	Section 9 extends it with the API suite

1.5 How this document was produced

Nothing here was taken on trust. The API has no published specification, no Swagger or OpenAPI endpoint and no written contract, so the surface was reconstructed from three independent sources and then confirmed against the running system:

1. **Traffic capture.** A headless browser walked every customer and administrator screen while all XHR and `fetch` traffic was recorded — 117 calls, giving real request bodies, real responses and real headers.
2. **Client bundle analysis.** The three JavaScript bundles (`index` , `orderNow` , `AdminRoutes`) were read to extract the complete API client object, which lists every path the front end can call — including those no screen reached during the walk.
3. **Direct probing.** Each endpoint was then called directly to confirm its method, parameters, clamping behaviour and error responses. Parameter names the minified bundle obscured were resolved by comparing result counts against the unfiltered total.

> **Confidence**
>
> Every status code, limit, field name and message quoted in sections 3, 4, 5 and 10 was observed directly. Where behaviour is specified but was not exercised — chiefly writes that would alter shared data — it is marked **NOT EXERCISED** and the reason given.

> **Two areas were deliberately left unexercised**
>
> `POST /api/checkout` places a real order against shared seed data, and bursting `POST /api/auth/login` trips the brute-force throttle that locks the single shared QA customer account for the whole team. Both are specified in full and have test cases, but those cases are gated behind `RUN_DESTRUCTIVE=1` and `RUN_THROTTLE=1` respectively, exactly as the UI suite gates their browser equivalents.

2 System under test

2.1 Architecture

North Pharmacy is a single-page React application served as static assets, talking to an ASP.NET Core JSON API on the same origin. There is no separate API host, no API gateway and no versioning segment in the path. The evidence for the stack is direct: responses carry `application/problem+json` bodies in the ASP.NET Core validation shape, the session cookie value is a Data Protection payload (the `CfDJ8` prefix), and failures surface a W3C `traceId`.

```
Browser (React SPA, /assets/*.js)
  |
  |  same-origin fetch, credentials: same-origin
  |  X-Requested-With: NorthPharmacy
  v
IIS  ->  ASP.NET Core API  ->  SQL Server
         /api/*                catalogue 17,453 products
                               orders, prescriptions, content, settings
         /uploads/*            product photos, manufacturer logos
         /api/prescriptions/{id}/file   prescription scans (access controlled)
```

Three consequences matter for testing. First, because the SPA and the API share an origin, there is no CORS preflight in normal operation and no bearer token — the session is a cookie. Second, because there is no version segment, a contract change is immediately breaking; contract tests are therefore the highest-value cases in this plan. Third, the product detail *page* is broken in this build while the product detail *endpoint* is healthy, which is only visible from this layer (see finding F-1 in section 10).

2.2 Environment

Base URL	`https://202.4.101.118:1165`, overridable as `BASE_URL`
TLS	Self-signed certificate. Clients must skip verification — an environment artefact per NP-QA-TC-001 §2.5, not a defect
Protocol	HTTP/1.1. HTTP/2 is not negotiated
Catalogue size	17,453 products, 80 manufacturers, 11 categories, 6 departments, 64 districts
Orders seeded	50 across six statuses (21 Pending, 6 Confirmed, 3 Packed, 3 Shipped, 4 Delivered, 13 Cancelled)
Outlets	Central, Indoor, Outdoor
Payment methods	Cash on Delivery only

> **One shared account per role**
>
> The environment issues exactly one QA customer and one administrator. Tests that mutate cart, wishlist or session state cannot run in parallel against it, and this is why the Playwright suite pins `workers: 1`. The API suite inherits the same constraint until per-worker accounts are provisioned — see section 9.5.

2.3 Accounts

ROLE	MOBILE	PASSWORD	NOTES
Customer	`01784066929`	`123Emsl!@#`	Masud Rana, id 3032. Six orders, lifetime spend 408.12

ROLE	MOBILE	PASSWORD	NOTES
Administrator	`01700000000`	`NorthAdmin@2026`	Store Admin, role `Admin`
Unregistered	`01999999999`	`NotAUser123!`	For negative authentication cases

Credentials belong in CI secrets as `TEST_USER`, `TEST_PASSWORD`, `ADMIN_USER`, `ADMIN_PASSWORD`. The values above are the shared QA accounts issued for this environment only and are reproduced here because the document is already environment-specific.

2.4 Reference data used by the test cases

ITEM	IDENTIFIER	WHY IT IS USED
Napa 500 mg Tablet	`4b588f69-008b-4e62-b98c-3f85c67199b5`	Non-prescription, in stock, price 1.20. The default happy-path product
Dps 10 cc/ml Surgical	`dda0268f-8fbc-4fd0-be63-b27019268d64`	Prescription-required, price 8.02. Used for Rx gating cases
Maxima 40 mg Injection	`5dd09c43-31fc-4c46-b1cb-ab4fd6aa3ff8`	Stock 1, so `maxQty` collapses to 1. Used for stock-boundary cases
Incepta Pharmaceutical Ltd	`d5111ffc-6a08-4e21-a885-393da7c38edd`	491 products. Used for the `mfr` filter
Order NP-104027	`104027`	Pending, owned by the QA customer, `canCancel: true`
Prescription 1006	`1006`	Approved, owned by a *different* customer. The authorisation fixture

2.5 Store rules the server enforces

These come from `GET /api/store/config` and are the source of truth for the calculation cases in module API-CHK. A test must read them rather than hard-code them, so that a settings change shows up as a configuration difference and not as a false failure.

RULE	VALUE	FIELD
Delivery inside home district	60.00	`deliveryChargeInside` (`homeDistrict` = Dhaka)
Delivery elsewhere	120.00	`deliveryChargeOutside`
Free delivery threshold	2,000.00	`freeDeliveryThreshold`
Maximum quantity per product	200	`maxQtyPerItem`, further capped per product by stock
Hotline	01330770000	`hotline`

3 API conventions

These conventions hold across the whole surface. Every case in section 7 assumes them, so a change to anything in this section invalidates a large part of the catalogue at once — which is why module API-CNV tests the conventions themselves rather than any single endpoint.

3.1 Transport and content types

Scheme	HTTPS only, self-signed certificate
Request body	`application/json; charset=utf-8`, or `multipart/form-data` for uploads
Response body	`application/json; charset=utf-8` on success; `application/problem+json` on error; `application/pdf` or an image type for prescription files; `text/csv` for report exports
Accept	The client sends `application/json`. The API does not vary on it
Caching	Authenticated responses carry `Cache-Control: no-cache,no-store`, `Pragma: no-cache` and a 1970 `Expires`
Compression	Not negotiated in this build. A 17 KB catalogue page is sent uncompressed — see finding F-8

3.2 Authentication

Authentication is a session cookie, not a token. `POST /api/auth/login` returns the user object and sets:

```
Set-Cookie: np_web_auth=CfDJ8I48h8phHTNCuRfQqDR_Y6GLASsGoyflJ6Ek...;
            expires=Sun, 08 Nov 2026 05:53:28 GMT;
            path=/; secure; samesite=lax; httponly
```

ATTRIBUTE	VALUE	ASSESSMENT
`HttpOnly`	set	GOOD Not readable from JavaScript, so XSS cannot exfiltrate the session
`Secure`	set	GOOD Never sent over plain HTTP
`SameSite`	`Lax`	GOOD Blocks cross-site POST; combined with 3.3 this is defence in depth
`Path`	`/`	Expected for a same-origin SPA
Lifetime	14 days	With `rememberMe: true`. Session-scoped otherwise

`GET /api/auth/me` is the session probe. It answers `200 {"user": null}` when anonymous — not 401 — so the SPA can boot without an error path. Every other protected endpoint answers `401` when anonymous.

3.3 The CSRF guard header — required on every call

Any request without this header is rejected with 400

```
X-Requested-With: NorthPharmacy
```

This is the single most important fact for anyone writing a client against this API. A request that omits the header — however well formed, however well authenticated — returns:

```
HTTP/1.1 400
Content-Type: application/json; charset=utf-8

{"title":"Request rejected.","status":400}
```

Note that this response is *not* in the RFC 7807 shape used by every other error, and it carries no `traceId`. It is produced by middleware ahead of the MVC pipeline. The guard relies on the fact that a custom request header cannot be set by a cross-origin form post or image tag without a CORS preflight the server will refuse, so it is a legitimate header-based CSRF defence. The value is a fixed literal, not a per-session token.

Testing consequences:

- Every request in the API suite must send the header. The suite sets it once in a shared request context (section 9.2) rather than per call.
- Its *absence* must itself be tested — cases API-CNV-TC-003 to 006 assert that state-changing endpoints reject a request without it.
- A 400 with the body `{"title":"Request rejected."}` during an unrelated test almost always means a missing header, not a validation failure. This has been the single biggest source of confusion when writing clients against this API.

3.4 Error model

Apart from the CSRF rejection above, errors follow RFC 7807 as emitted by ASP.NET Core. Three distinct shapes occur, and a test asserting on an error must know which one to expect.

Field validation — errors keyed by field name

```
HTTP/1.1 400
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": { "phone": ["Enter a valid Bangladeshi mobile number (01XXXXXXXXX)."] },
  "traceId": "00-a177b753832cdb52146f8ade3072b530-8afff605e1063f35-00"
}
```

Form-level rule — errors keyed by the empty string

```
HTTP/1.1 400
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": { "": ["Maximum 200 per product."] },
  "traceId": "..."
}
```

Bare problem — no errors object

```
HTTP/1.1 404
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
  "title": "Not Found",
  "status": 404,
  "traceId": "..."
}
```

How the client reads these

The SPA maps `errors[""]` and any key beginning with `$` to a form-level banner, and every other key to the matching input, lower-cased. It replaces the server text entirely for 401, 404, 429 and 5xx. A test must therefore assert on the *API* response, never on the message the UI displays — they are frequently different strings.

3.5 Status code usage

CODE	USED FOR	OBSERVED EXAMPLE
`200`	Success with a body	All reads; also writes that return the new state, such as cart mutations
`204`	Success, no body	`POST /api/auth/logout` — the only 204 observed
`400`	Validation failure, or a missing CSRF header	Quantity 0, duplicate phone, weak password
`401`	Not signed in. Body is empty	`GET /api/orders` with no cookie
`403`	Signed in, wrong role	Customer calling `GET /api/admin/orders`
`404`	Not found, *or* found but not yours	Another customer's order; an unknown product id
`429`	Rate limited	The tenth `POST /api/auth/register` within the window

404 instead of 403 for other people's records is deliberate and correct

Requesting another customer's order returns `404`, not `403`. This is the right choice: a 403 would confirm that the record exists, which lets an attacker enumerate order numbers. Tests must assert `404` here and must not be "corrected" to 403.

3.6 Pagination

Collection endpoints return an envelope. The catalogue envelope is:

```
{ "title": "All products", "subtitle": null,
  "items": [ ... ], "total": 17453, "page": 1, "pageSize": 24, "totalPages": 728,
  "categoryFacets": [ { "key": "...", "name": "...", "count": 7539 } ],
  "manufacturerFacets": [ ... ], "generic": null, "manufacturer": null }
```

Both paging parameters are **clamped rather than rejected**, which is a design choice worth knowing before writing boundary cases:

REQUEST	EFFECTIVE	OBSERVED
`pageSize=1`	12	Returned 12 items, `pageSize: 12` echoed
`pageSize=24`	24	Honoured
`pageSize=500`	60	Returned 60 items — hard upper bound
`page=99999`	728 (last)	Returned the final 5 items, not an empty page
`page=0` / negative	1	Clamped to the first page

So `pageSize` is clamped to [12, 60] and `page` to [1, `totalPages`]. A client that walks pages by incrementing until an empty page arrives will loop forever; it must compare against `totalPages`. Admin collections use the same envelope plus a `counts` object, and do not accept `pageSize` at all.

3.7 Rate limiting

Rate limiting exists but is applied narrowly. It was characterised by sending measured bursts and recording the request index at which the response changed.

ENDPOINT	RESULT	EVIDENCE
`POST /api/auth/register`	LIMITED ~10 per minute	Requests 1–9 returned 400; requests 10, 11 and 12 returned 429. Cleared within a minute
`POST /api/auth/login`	LIMITED	Brute-force throttling is asserted by NP-LOG-TC-010. **Not re-tested here** — it locks the shared account
`POST /api/cart/quote`	NOT LIMITED	12 consecutive requests, all 200
`GET /api/catalog/categories`	NOT LIMITED	30 rapid requests, zero 429

The 429 response carries **no `Retry-After` header** and no `RateLimit-*` headers, so a client cannot know how long to wait; the SPA hard-codes the advice “wait a minute”. This is recorded as finding F-6.

3.8 Security headers

Present on every response, including errors:

HEADER	STATUS	NOTE
`Strict-Transport-Security: max-age=31536000`	PRESENT	One year. No `includeSubDomains`
`X-Content-Type-Options: nosniff`	PRESENT	
`X-Frame-Options: DENY`	PRESENT	
`Referrer-Policy: strict-origin-when-cross-origin`	PRESENT	
`Content-Security-Policy`	ABSENT	Finding F-7
`X-Powered-By`	EMPTY	Sent twice, both blank. Harmless but untidy — finding F-7

CORS is configured for the single origin: responses carry `Access-Control-Allow-Origin: https://202.4.101.118:1165` and `Access-Control-Allow-Credentials: true`. Because the SPA is same-origin this

never engages in normal use, but it does mean an automated client sending a foreign `Origin` is still served — the CSRF guard in 3.3, not CORS, is what actually protects writes.

4 Endpoint reference — storefront

Forty-two endpoints. The *Auth* label on each block means: **none** — works anonymously; **customer** — requires the session cookie, 401 without it. Every request additionally requires the `X-Requested-With: NorthPharmacy` header from section 3.3; it is not repeated in each block.

4.1 Store

GET `/api/store/config` — AUTH: NONE

Store-wide settings. The SPA fetches this once at boot and every delivery and quantity rule derives from it. **Test cases must read their expectations from here** rather than hard-coding 60, 120 or 2000.


```
{ "name": "North Pharmacy", "hotline": "01330770000", "hotlineHours": null, "email": null,
  "address": "Cemex Iconic Tower, Plot No. 24/1-A (Ka) ... Dhaka-1207",
  "social": {}, "homeDistrict": "Dhaka",
  "deliveryChargeInside": 60, "deliveryChargeOutside": 120,
  "freeDeliveryThreshold": 2000, "maxQtyPerItem": 200,
  "paymentMethods": [ { "code": "COD", "label": "Cash on Delivery", "walletNumber": null } ],
  "departments": [ { "key": "medicines", "name": "Medicines",
                     "categoryKeys": ["tablets-capsules", "syrups-liquids", ...] }, ... ] }
```


Observed 0.9 KB, 2 ms. Mirrors `GET /api/admin/settings`, which is the writable view of the same data — cases API-AST-TC-005 to 008 assert the two stay in step.

GET `/api/store/content` — AUTH: NONE

All editorial copy: hero slides, top-bar messages, static page bodies, FAQs. Returned as nested objects keyed by page. Strings may contain placeholders such as `{freeDeliveryAbove}`, `{homeDistrict}`, `{deliveryInside}`, which the *client* substitutes from `/api/store/config`.

> Because substitution happens client-side, an unresolved placeholder is invisible to a response-body assertion. Case API-STO-TC-005 therefore asserts that every `{token}` appearing in content has a matching key in config.

Observed 7.7 KB, 3 ms. Writable through the four `/api/admin/content/*` endpoints.

4.2 Catalogue

GET `/api/catalog/products` — AUTH: NONE

The main catalogue query: search, filter, facet and page in one call. The busiest and heaviest endpoint in the system, and the one that governs load behaviour (NP-LT-001 §5).

PARAMETER	TYPE	MEANING AND OBSERVED BEHAVIOUR
`q`	string	Free text over brand and generic. `q=napa` returns 33. Sets `title` to `Results for "napa"`
`cat`	category key	Category slug, *not* `category`. `cat=tablets-capsules` returns 7,539
`dept`	department key	`dept=medicines` returns 11,561
`mfr`	GUID	Manufacturer id, *not* name. Returns 491 for Incepta
`generic`	GUID	Generic id
`inStock`	`true`	Narrows 17,453 to 3,084
`offers`	`true`	Discounted only. Returns 2; `title` becomes `Offers`
`noPhoto`	`true`	Products still showing placeholder art
`sort`	enum	omitted (most popular), `relevance`, `price-asc`, `price-desc`, `name`
`page`	int	Clamped to [1, `totalPages`]
`pageSize`	int	Clamped to [12, 60]. The SPA always sends 24

Unknown parameters are silently ignored

`?category=tablets-capsules` and `?manufacturer=<guid>` both return the full unfiltered 17,453 with HTTP 200. A client using the wrong name gets a plausible-looking success and wrong data. This is finding F-3; cases API-CAT-TC-021 to 024 pin the behaviour so that a future decision to reject unknown parameters is caught.

Response envelope and one item:


```
{ "title": "All products", "subtitle": null, "total": 17453,
  "page": 1, "pageSize": 24, "totalPages": 728,
  "items": [ { "id": "dda0268f-8fbc-4fd0-be63-b27019268d64", "name": "Dps",
      "slug": "dps-10-cc-ml-surgical", "generic": "Disposable Syringe",
      "genericId": "211f7032-...", "formation": "Surgical", "strength": "10 cc/ml",
      "subtitle": "10 cc/ml · Surgical", "packUnit": "piece",
      "manufacturer": "Nipro Jmi Medical Ltd.", "manufacturerId": "9de21329-...",
      "categoryKey": "devices", "categoryName": "Surgical & Devices",
      "tone": "teal", "art": "device",
      "mrp": 8.02, "price": 8.02, "discountPercent": 0, "hasDiscount": false,
      "inStock": true, "lowStock": false, "stockLeft": null, "maxQty": 200,
      "requiresPrescription": true, "isFeatured": true, "imageUrl": null } ],
  "categoryFacets":     [ { "key": "tablets-capsules", "name": "...", "count": 7539 } ],
  "manufacturerFacets": [ { "key": "bfb229bf-...", "name": "...", "count": 841 } ],
  "generic": null, "manufacturer": null }
```

Observed 17.3 KB, 15 ms unfiltered — five to fifteen times heavier than any other read. `stockLeft` is null unless `lowStock` is true, so exact stock is not disclosed for healthy lines.

GET /api/catalog/products/{id}

AUTH: NONE

`{id}` is the product **GUID**. A slug returns 404.

```
{ "product": { ...same 26 fields as a list item... },
  "description": "...", "generic": { ... },
  "alternatives": [ 8 items ], "related": [ 10 items ] }
```

REQUEST	STATUS	NOTE
`/4b588f69-...-3f85c67199b5`	200	12.1 KB, 3 ms
`/napa-500-mg-tablet` (slug)	404	Slugs are not accepted, despite appearing in the page URL
`/00000000-0000-...-000000000000`	404	RFC 7807 Not Found
`/not-a-guid`	404	Empty body, no problem document

> **This endpoint is healthy — the page that uses it is not**
>
> DEFECTS.md records every `/p/:id/:slug` page returning HTTP 502, failing 21 UI cases in module A4. This endpoint answers 200 with complete data in 3 ms. The fault is therefore in page delivery, not in the catalogue service — a conclusion only reachable from the API layer. See finding F-1.

GET /api/catalog/suggest

AUTH: NONE

Type-ahead for the search box. Parameter `q`. Returns a bare array (no envelope) of trimmed product records: `id`, `slug`, `name`, `sub`, `manufacturer`, `price`, `inStock`, `art`, `tone`, `imageUrl`.

Empty `q` returns `[]` with 200. `q=' OR 1=1--` returns `[]` with 200 — parameterised, no injection. Observed 2.0 KB, 5 ms.

GET /api/catalog/home

AUTH: NONE

Home-page merchandising: `bestSellers`, featured and department strips, each a list of the same product shape. Observed 19.3 KB, 5 ms — the largest single read on the storefront.

GET `/api/catalog/categories` — AUTH: NONE

The 11 categories with live counts:

```
[ { "key": "tablets-capsules", "name": "Tablets & Capsules",
    "blurb": "Prescription and OTC oral medicines", "tone": "blue", "art": "capsule",
    "productCount": 7539, "inStockCount": 1637 }, ... ]
```

Case API-CAT-TC-009 cross-checks each `productCount` against `total` from `/api/catalog/products?cat={key}`; they must agree.

GET `/api/catalog/manufacturers` — AUTH: NONE

Array of 80 manufacturers with `id`, `name`, `logoUrl` and a product count. The `id` is what `?mfr=` expects. 10.3 KB, 2 ms.

GET `/api/catalog/districts` — AUTH: NONE

Array of 64 district names, alphabetical, Bagerhat first. Drives address and checkout selects. Case API-CAT-TC-014 asserts the count is 64 and that `homeDistrict` from store config is a member. 0.7 KB, 1 ms — the cheapest endpoint, and so the natural health-check target for the load plan.

4.3 Authentication

POST `/api/auth/login` — AUTH: NONE

```
Request   { "phone": "01784066929", "password": "123Emsl!@#", "rememberMe": true }
Response  200  { "id": 3032, "fullName": "Masud Rana", "phone": "01784066929",
                 "email": null, "role": "Customer" }
          + Set-Cookie: np_web_auth=...; secure; samesite=lax; httponly
```

CONDITION	STATUS	BODY
Wrong password	400	`errors: { "": ["That mobile number and password don't match."] }`
Unknown number	400	Same message — correctly does not reveal whether the account exists
`password` missing	400	`errors: { "password": ["Enter your password"] }`
No CSRF header	400	`{"title":"Request rejected.","status":400}`
Repeated failures	429	**NOT EXERCISED** — locks the shared account. Covered by NP-LOG-TC-010

`rememberMe: true` yields a 14-day cookie; omitted or false yields a session cookie. `role` is `Customer` or `Admin` and is what gates the admin surface.

POST `/api/auth/register` — AUTH: NONE

```
Request   { "fullName": "...", "phone": "01XXXXXXXXX", "email": "optional",
            "password": "...", "confirm": "..." }
```

Validation messages observed directly:

INPUT	FIELD KEY	MESSAGE
Name shorter than 2 words / too short	`fullName`	Enter your full name
`phone: "123"`	`phone`	Enter a valid Bangladeshi mobile number (01XXXXXXXXX).
Number already registered	`phone`	An account with this number already exists. Try signing in.
`password: "Abcd1234!"`	`password`	This password is too common and easy to guess. Choose another.

Validation short-circuits, and the endpoint is rate limited

Only the first failing field is reported, so a payload with three faults returns one error. A test asserting “all errors present” will fail for the wrong reason. Separately, the tenth call within roughly a minute returns 429 — a negative-path suite that fires a dozen invalid registrations in a loop will see spurious failures from request ten onward. Cases API-AUT-TC-018 to 020 account for this.

GET `/api/auth/me` — AUTH: NONE

Session probe. `200 {"user": null}` anonymous; `200 {"user": {...}}` signed in. Never 401. 1 ms, <0.1 KB — the cheapest authenticated-state check available.

POST `/api/auth/logout` — AUTH: CUSTOMER

No body. Returns **204** — the only 204 in the API — and expires the cookie. Case API-AUT-TC-012 asserts that a protected call with the same cookie afterwards returns 401.

GET /api/auth/forgot/channels — AUTH: NONE

Lists the recovery channels available. **Observed:** `{"channels": []}` — an empty array, which is how the feature is switched off.

> **Password recovery is disabled in this build**
>
> An empty `channels` array makes the UI show "Online reset isn't available right now" and permanently disable the Send code button. All eight UI cases NP-FGT-TC-001 to 008 are **Blocked** for this reason. The API cases mirror that: API-AUT-TC-021 asserts the empty array as the current expected state, and API-AUT-TC-022 and 023 for `forgot/send` and `forgot/reset` are marked Blocked rather than Failed. Whether this is intentional for the build is still an open question for the business.

POST /api/auth/forgot/send — AUTH: NONE

Body `{ "channel": "...", "contact": "..." }`. BLOCKED No channel exists to select, so the endpoint cannot be meaningfully called.

POST /api/auth/forgot/reset — AUTH: NONE

Body carries the code and the new password. BLOCKED No code can be issued.

4.4 Cart

> **The cart is server-side and requires a session**
>
> Every cart endpoint returns **401 when anonymous** — verified. An anonymous shopper's cart lives only in the browser, which is why `POST /api/cart/merge` exists: the SPA replays the local cart into the server cart at sign-in. Any test touching the cart must authenticate first.

GET /api/cart — AUTH: CUSTOMER

```
{ "items": [ { "productId": "dda0268f-...", "quantity": 1 } ], "notices": [] }
```

`notices` carries server-raised warnings — a line that fell out of stock, or a quantity reduced to available stock. It is the field most likely to be forgotten by a client, so API-CRT-TC-014 asserts it is always present, even when empty.

PUT /api/cart/items/{productId}

AUTH: CUSTOMER

Sets an absolute quantity — it is not an increment. Creates the line if absent. Returns the whole cart.

```
Request   { "quantity": 1 }
Response  200  { "items": [ { "productId": "dda0268f-...", "quantity": 1 } ], "notices": [] }
```

QUANTITY	STATUS	MESSAGE IN `ERRORS[""]`
`0`	400	Minimum 1 per product.
`-1`	400	Minimum 1 per product.
`9999`	400	Maximum 200 per product.
Unknown product GUID	404	This product is no longer available.

The 200 cap matches `maxQtyPerItem` from store config. Per-product stock narrows it further: Maxima has `stockLeft: 1` and `maxQty: 1`, so quantity 2 must also be rejected — API-CRT-TC-008.

DELETE /api/cart/items/{productId}

AUTH: CUSTOMER

Removes one line. Returns the updated cart. Verified 200.

DELETE /api/cart

AUTH: CUSTOMER

Empties the cart. Returns `{"items":[],"notices":[]}`. Verified 200.

POST /api/cart/merge

AUTH: CUSTOMER

Body `{ "items": [ { "productId": "...", "quantity": n } ] }`. Called once immediately after sign-in to fold the anonymous browser cart into the account cart. Whether quantities are summed or replaced on collision is the contract question here — API-CRT-TC-016 pins it.

POST `/api/cart/quote` — AUTH: NONE

Prices a basket without creating an order — subtotal, discount, delivery charge, total. Works anonymously, and is not rate limited.

```
Request   { "items": [ { "productId": "...", "quantity": 2 } ], "district": "Dhaka" }
```

The highest-value endpoint in this plan

Quote is where every money rule can be proved without placing an order: 60 inside Dhaka, 120 elsewhere, free above 2,000, and the boundary either side of the threshold. In the browser these need a full checkout; here each is one request. Module API-CHK devotes eleven cases to it, and all of them are non-destructive.

4.5 Wishlist

GET `/api/wishlist` — AUTH: CUSTOMER

Full product records for saved items. `[]` when empty.

GET `/api/wishlist/ids` — AUTH: CUSTOMER

Just the GUIDs, so the SPA can fill heart icons on listing pages without a second heavy payload. API-WSH-TC-005 asserts the two endpoints agree.

POST `/api/wishlist/{productId}/toggle` — AUTH: CUSTOMER

No body. Adds if absent, removes if present — so it is not idempotent, and calling it twice must return to the starting state (API-WSH-TC-003). Verified 200.

4.6 Account

GET `/api/account/summary` — AUTH: CUSTOMER


```
{ "user": { "id": 3032, "fullName": "Masud Rana", "phone": "...", "email": null,
            "role": "Customer" },
  "memberSince": "2026-10-24T10:31:23.329Z",
  "orderCount": 6, "activeOrderCount": 6, "lifetimeSpend": 408.12,
  "prescriptionCount": 0, "wishlistCount": 0,
  "recentOrders": [ { "id": 104027, "orderNo": "NP-104027", "status": "Pending",
                      "paymentMethod": "COD", "paymentStatus": "Unpaid",
                      "total": 68.02, "productCount": 1, "pieceCount": 1,
                      "itemNames": ["Dps"] } ] }
```


> **The API has a field the UI is missing**
>
> DEFECTS.md notes that NP-ACC-TC-013 expects a "Member since" date that the account page does not display. The API returns `memberSince` correctly. So this is a presentation gap, not missing data — a one-line fix on the client. Finding F-2.

PUT `/api/account/profile` — AUTH: CUSTOMER

Updates name and e-mail. NOT EXERCISED — would mutate the shared account. Gated behind `RUN_DESTRUCTIVE=1`.

POST `/api/account/password` — AUTH: CUSTOMER

Changes the password. NOT EXERCISED — changing the shared QA password locks out the whole team and every other suite. Documented, gated, and run only against a disposable environment.

GET `/api/account/addresses` — AUTH: CUSTOMER

Array of saved addresses. Currently `[]` for the QA customer.

POST `/api/account/addresses` — AUTH: CUSTOMER


```
{ "recipientName": "...", "phone": "01XXXXXXXXX",
  "addressLine": "...", "area": "...", "district": "Dhaka" }
```


`district` must be one of the 64 from `/api/catalog/districts`; API-ACC-TC-011 asserts an unknown district is rejected.

POST `/api/account/addresses/{id}/default` — AUTH: CUSTOMER

Promotes one address to default. API-ACC-TC-014 asserts exactly one address carries the default flag afterwards.

DELETE `/api/account/addresses/{id}` — AUTH: CUSTOMER

Removes an address. API-ACC-TC-016 asserts another customer's address id returns 404, not 403 or 204.

4.7 Prescriptions

GET `/api/prescriptions` — AUTH: CUSTOMER

The signed-in customer's uploads only. `[]` for the QA customer.

POST `/api/prescriptions` — AUTH: CUSTOMER

`multipart/form-data` with parts `file` (required), `patientName` and `note`.

RULE	CLIENT-SIDE VALUE — *SERVER ENFORCEMENT MUST BE PROVED SEPARATELY*
Accepted types	`.jpg .jpeg .png .webp .pdf`
Maximum size	8 MB (8,388,608 bytes)
Empty file	Rejected with "That file is empty."

> **These limits were read from the client bundle, not proved on the server**
>
> The extension list and the 8 MB cap are enforced in JavaScript before upload. A client that posts directly to the API bypasses that check entirely. Whether the server repeats the validation is **unproven and is the highest-priority gap in this plan**. The suite already carries the fixtures to settle it — `oversized-9mb.jpg`, `payload.exe`, `renamed-exe.jpg` and `not-a-prescription.txt` — and cases API-SEC-TC-012 to 016 post each one directly. A pharmacy accepting arbitrary uploads is a serious exposure, so these run before any release sign-off.

GET `/api/prescriptions/{id}/file` — AUTH: CUSTOMER / ADMIN

Streams the stored scan. Returns the original content type, for example `application/pdf`.

CALLER	STATUS	RESULT
Anonymous	401	Empty body
Customer, someone else's prescription	404	Problem document. Existence not disclosed
Administrator	200	558,571 bytes, `application/pdf`

> **Verified: medical records are properly scoped**
>
> This was tested directly against prescription 1006, which belongs to a different customer. The QA customer received 404 and the administrator received the file. For a pharmacy, this is the single most sensitive authorisation boundary in the product, and it holds. Cases API-SEC-TC-005 to 008 keep it that way.

4.8 Checkout and orders

GET `/api/checkout/options` — AUTH: CUSTOMER

Everything the checkout form needs in one call: `addresses`, `prescriptions`, `districts` (all 64), and the payment methods.

POST `/api/checkout` — AUTH: CUSTOMER

DESTRUCTIVE — NOT EXERCISED Places a real order against shared seed data. Specified from the client source; cases are gated behind `RUN_DESTRUCTIVE=1`.

```
{ "items": [ { "productId": "...", "quantity": 2 } ],
  "addressId": 12,                      // or null, and then the five inline fields:
  "recipientName": "...", "phone": "...", "addressLine": "...",
  "area": "...", "district": "Dhaka", "saveAddress": true,
  "paymentMethod": "COD",
  "paymentRef": null,                   // required when paymentMethod != COD
  "prescriptionId": null,               // required when any line needs a prescription
  "note": "" }

Response  { "id": 104027, "orderNo": "NP-104027", ... }
```

Either `addressId` or the five inline address fields must be supplied, never neither. If any line has `requiresPrescription: true` the order must carry a `prescriptionId` — API-CHK-TC-013 asserts the server rejects an order that omits it, because that gate is what makes the pharmacy lawful rather than merely tidy.

GET /api/orders

AUTH: CUSTOMER

A bare array — **not paginated**, unlike every other collection. Newest first. Each entry: `id`, `orderNo`, `createdOn`, `status`, `paymentMethod`, `paymentMethodLabel`, `paymentStatus`, `total`, `productCount`, `pieceCount`, `itemNames`.

Unbounded growth is a latent performance and contract risk: a customer with 800 orders receives all 800. Raised as finding F-5 and exercised by NP-LT-001.

GET /api/orders/{id}

AUTH: CUSTOMER


```
{ "id": 104027, "orderNo": "NP-104027", "createdOn": "2026-10-24T16:57:46.183Z",
  "status": "Pending", "canCancel": true,
  "customerName": "Masud Rana", "phone": "01784066929",
  "addressLine": "House 7, Road 2, Mohammadpur", "area": null, "district": "Dhaka",
  "subtotal": 8.02, "discount": 0, "deliveryCharge": 60, "total": 68.02,
  "paymentMethod": "COD", "paymentMethodLabel": "Cash on Delivery",
  "paymentStatus": "Unpaid", "paymentRef": null, "walletNumber": null,
  "customerNote": null, "prescription": null,
  "items": [ { "productId": "...", "productName": "Dps", "genericName": "Disposable Syringe",
               "strength": "10 cc/ml", "formation": "Surgical", "manufacturer": "...",
               "unitPrice": 8.02, "quantity": 1, "lineTotal": 8.02,
               "requiresPrescription": true } ],
  "history": [ { "status": "Pending", "note": null, "createdOn": "..." } ],
  "productCount": 1, "pieceCount": 1 }
```


Verified authorisation: order 104000 and order 1, both belonging to other customers, each returned `404`. `canCancel` is the server's own judgement and the client must obey it rather than deciding from `status`. `subtotal + deliveryCharge − discount` must equal `total` — asserted by API-ORD-TC-006 on every order returned.

POST /api/orders/{id}/cancel

AUTH: CUSTOMER

Body `{ "reason": "..." }`. DESTRUCTIVE Cancelling consumes a seeded Pending order. API-ORD-TC-009 asserts an order with `canCancel: false` is refused, and API-ORD-TC-011 that another customer's order returns 404.

POST /api/orders/{id}/reorder

AUTH: CUSTOMER

Refills the cart from a past order. No body. DESTRUCTIVE — overwrites the shared cart. API-ORD-TC-013 asserts that lines now out of stock surface in `notices` rather than failing the whole call.

4.9 Newsletter

`POST` **`/api/newsletter`** — **AUTH: NONE**

Footer subscription. Body carries the e-mail address. Anonymous and unauthenticated, so API-SEC-TC-019 checks it for enumeration leakage — the response must not differ between an address already subscribed and a new one.

5 Endpoint reference — administration

Thirty-one endpoints under `/api/admin/*`. All require a session whose `role` is `Admin`.

> **Role separation verified across the whole surface**
>
> Anonymous callers receive `401`; a signed-in *customer* receives `403` — confirmed on `GET /api/admin/summary`, `GET /api/admin/orders`, `GET /api/admin/prescriptions` and `POST /api/admin/prescriptions/1006/review`. The last of these is the important one: a customer cannot reach the prescription review action even by calling it directly. Module API-SEC walks every admin route with a customer cookie to keep this true.

Admin collections share one envelope — `items`, `total`, `page`, `totalPages`, plus a `counts` object giving the size of each tab. Page size, however, is **not uniform**, and a client cannot assume it:

COLLECTION	PAGE SIZE	PAGESIZE PARAM	OBSERVED
`/api/admin/orders`	20 by default	HONOURED	Clamped to a minimum of 10; `pageSize=50` returned 50. `pageSize` echoed in the envelope
`/api/admin/prescriptions`	fixed	IGNORED	No `pageSize` field in the envelope at all
`/api/admin/inventory`	fixed 25	IGNORED	25 items whatever is requested
`/api/admin/customers`	fixed 25	IGNORED	25 items; 65 customers over 3 pages
`/api/admin/manufacturers`	fixed 24	IGNORED	24 items over 13 pages

> Only `/api/admin/orders` honours `pageSize`, and only it reports the page size it used. The other four silently ignore the parameter and omit the field, so a client has no way to discover the page size except by counting the items it received. Cases API-AOR-TC-001, API-AIN-TC-002, API-ACU-TC-002 and API-ARX-TC-002 pin the current behaviour per collection rather than assuming a shared rule.

5.1 Dashboard

GET `/api/admin/summary` — AUTH: ADMIN

The four sidebar badge counts, polled by every admin screen.

```
{ "toConfirm": 21, "inProgress": 6, "prescriptionsToReview": 0, "lowStock": 772 }
```

Each number must agree with the matching list endpoint's `counts`; API-ADM-TC-003 cross-checks all four.

GET `/api/admin/dashboard` — AUTH: ADMIN

Parameter `days` (the UI offers 7, 30, 90). Seventeen top-level keys:

```
generatedAt, days, from, to, lastLoginOn, attention, ordersToday, salesToday,
customers, current, previous, daily, statuses, topOnline, topPharmacy,
activities, catalog
```

`current` and `previous` are matched period totals, so the dashboard can show a change. API-ADM-TC-006 asserts that `from` and `to` actually span `days`, and API-ADM-TC-007 that `daily` has exactly `days` entries — an off-by-one here silently misstates every trend on the page.

Dates are computed in `Asia/Dhaka`. A test running on a UTC CI agent near midnight will disagree with the server by one day unless it uses the same zone — API-ADM-TC-008 covers this explicitly.

5.2 Orders

GET `/api/admin/orders` — AUTH: ADMIN

Parameters `status`, `q`, `page` and — uniquely among the admin lists — `pageSize`. `status=active` is a pseudo-status for everything in progress.


```
{ "items": [ { "id": 104027, "orderNo": "NP-104027", "createdOn": "...",
      "customerName": "Masud Rana", "phone": "01784066929", "district": "Dhaka",
      "productCount": 1, "pieceCount": 1, "total": 68.02,
      "paymentMethod": "COD", "paymentMethodLabel": "Cash on Delivery",
      "paymentStatus": "Unpaid", "status": "Pending",
      "hasRx": true, "prescriptionStatus": null } ],
  "total": 50, "page": 1, "pageSize": 20, "totalPages": 3,
  "counts": { "Pending": 21, "Confirmed": 6, "Packed": 3,
              "Shipped": 3, "Delivered": 4, "Cancelled": 13 } }
```


A prescription-required order with no prescription decision

Order 104027 carries `hasRx: true` with `prescriptionStatus: null`, and the customer-side record shows `prescription: null`. So an order containing a prescription-only product exists in Pending with no prescription attached at all. Whether the server should have refused it at checkout is finding F-4; API-CHK-TC-013 is the case that settles it.

GET `/api/admin/orders/{id}` — AUTH: ADMIN

Full order including `items` and `history`, for any customer. API-AOR-TC-004 asserts it matches the customer-side view of the same order field for field, apart from the admin-only additions.

POST `/api/admin/orders/{id}/status` — AUTH: ADMIN

```
{ "status": "Confirmed", "note": "optional", "outletId": "99c43752-..." }
```

Statuses: `Pending`, `Confirmed`, `Packed`, `Shipped`, `Delivered`, `Cancelled`. DESTRUCTIVE

Two contract questions the cases must answer: whether an invalid transition (Delivered back to Pending) is refused — API-AOR-TC-008 — and whether every change appends to `history` — API-AOR-TC-009. The second matters because `history` is the audit trail a dispute relies on.

POST `/api/admin/orders/{id}/payment` — AUTH: ADMIN

Body `{ "paymentStatus": "Paid" }`. Observed values `Unpaid` and `Paid`. DESTRUCTIVE

GET `/api/admin/orders/{id}/outlets` — AUTH: ADMIN

Outlets that can fulfil this order — Central, Indoor, Outdoor. Feeds `outletId` on the status call.

5.3 Prescriptions

GET `/api/admin/prescriptions` — AUTH: ADMIN

Parameters `status` (`Pending`, `Approved`, `Rejected`, `all`) and `page`.

```
{ "items": [ { "id": 1006, "createdOn": "...",
      "customerName": "Rabby Islam", "customerPhone": "01988776656",
      "patientName": "Rabby Islam", "note": "Uploaded at checkout",
      "status": "Approved", "reviewNote": null,
      "reviewedBy": "Store Admin", "reviewedOn": "2026-10-24T05:59:00.932Z",
      "isImage": false, "originalFileName": "AKIJ_FINAL_LIST.pdf",
      "fileSize": 558571, "fileUrl": "/api/prescriptions/1006/file",
      "orderIds": [ 104021 ] } ],
  "total": 7, "page": 1, "totalPages": 1,
  "counts": { "Approved": 6, "Rejected": 1 } }
```

`reviewedBy` and `reviewedOn` are the pharmacist sign-off. They must be set together and must never be cleared — API-ARX-TC-007.

POST `/api/admin/prescriptions/{id}/review` — AUTH: ADMIN

```
{ "status": "Approved" | "Rejected", "note": "optional" }
```

Highest-priority case in the whole plan — API-ARX-TC-005

DEFECTS.md issue 6 records that in the UI, a prescription already in the Approved tab still offers an *enabled* Reject button, so a pharmacist's decision can be reversed by a second administrator with no conflict check. The UI case NP-RXA-TC-014 proves the button is enabled. It does not prove what the server does when that button is pressed, and that is the question that actually matters.

API-ARX-TC-005 answers it directly: POST a review of `Rejected` to a prescription already `Approved` and record the status code. If the server accepts it, the defect is a true server-side audit failure and is Critical. If the server returns 409 or 400, the defect is cosmetic and drops to Low. This single request changes the severity of a Critical open defect, so it is the first case to run once the destructive gate is opened against a disposable environment.

Confirmed: a customer-role session calling this endpoint receives `403`, so the exposure is limited to administrators.

DELETE `/api/admin/prescriptions/{id}` — AUTH: ADMIN

Permanently removes a prescription. **DESTRUCTIVE** API-ARX-TC-010 asserts that deleting one attached to an order is refused, or that the order's reference is handled rather than left dangling.

5.4 Inventory and products

GET `/api/admin/inventory` — AUTH: ADMIN

Parameters `view`, `q`, `outlet`, `page`.

VIEW	COUNT	MEANING
`low`	772	10 or fewer units left
`out`	144	None left but sold at the counter in the last 90 days
`nophoto`	3,067	In stock but still showing placeholder art
`offers`	2	Discounted
`featured`	3	Featured
`hidden`	1	Hidden from the storefront
`all`	17,453	Everything

```
{ "items": [ { "product": { ...26 catalogue fields... },
               "stock": 1, "soldAtPos": 350, "hasPhoto": false, "isHidden": false } ],
  "counts": { "low": 772, "out": 144, "nophoto": 3067, "offers": 2,
              "featured": 3, "all": 17453, "hidden": 1 },
  "outlets": [ { "id": "99c43752-...", "name": "Central Outlet" },
               { "id": "9f2d97f5-...", "name": "Indoor Outlet" },
               { "id": "0196b96d-...", "name": "Outdoor Outlet" } ] }
```

Reproduce DEFECTS.md issue 2 here, in one request

Admin product search finds nothing for brand names that the customer catalogue finds easily: `Napa` returns 0 in admin against 24 in the catalogue; `Dps` returns 0 against a product that plainly exists; generic names such as `Paracetamol` work in both. Case API-AIN-TC-004 compares `/api/admin/inventory?view=all&q=Napa` with `/api/catalog/products?q=Napa` and asserts the counts agree. It is a two-request test for a defect that currently needs two browser sessions, and it tells the developer immediately whether the fault is in the query or in the screen.

GET `/api/admin/products/{id}/content` — AUTH: ADMIN

Editable description, discount and visibility for one product.

PUT `/api/admin/products/{id}/content` — AUTH: ADMIN

Saves them. **DESTRUCTIVE** — changes what every customer sees. API-AIN-TC-008 asserts a negative or above-100 discount is rejected.

POST `/api/admin/products/{id}/image` — AUTH: ADMIN

`multipart/form-data`, part `file`. Client rules: PNG, JPG, WEBP or SVG, maximum 3 MB. As with prescriptions, server-side enforcement is unproven — API-SEC-TC-017.

SVG is accepted here but not for prescriptions. An SVG is an XML document that can carry script, so if these files are ever served inline from the same origin that is a stored-XSS vector. API-SEC-TC-018 uploads an SVG containing a script element and checks both whether it is accepted and what `Content-Type` and `Content-Disposition` it is served back with.

DELETE `/api/admin/products/{id}/image` — AUTH: ADMIN

Reverts the product to placeholder art. DESTRUCTIVE

5.5 Customers

GET `/api/admin/customers` — AUTH: ADMIN

Parameters `q` and `page`. Standard envelope without `counts`. Returns name, mobile, order count and lifetime spend.

This is a personal-data endpoint. API-SEC-TC-009 confirms a customer role receives 403, and API-ACU-TC-005 that no password hash, token or internal identifier appears in the payload.

5.6 Manufacturers

GET `/api/admin/manufacturers` — AUTH: ADMIN

Parameters `q`, `view`, `page`. Envelope adds `withLogo`, `all` and `homeMissing` counts.

POST `/api/admin/manufacturers/{id}/logo` — AUTH: ADMIN

Multipart logo upload, part `file`. Same 3 MB image rules.

DELETE `/api/admin/manufacturers/{id}/logo` — AUTH: ADMIN

Removes the logo. DESTRUCTIVE

5.7 Content

GET `/api/admin/content` — AUTH: ADMIN

Returns `home`, `pages`, `faqs` and `icons` — the editable form of `/api/store/content`.

PUT `/api/admin/content/home` — AUTH: ADMIN

```
{ "content": { ... }, "version": 7 }
```

> **Optimistic concurrency — the only place it appears**
>
> All three content saves carry a `version`. This is the API's one explicit lost-update defence, and it is worth proving: API-ACN-TC-004 reads the content, saves once, then saves again with the *stale* version and asserts the second save is refused. If it is not, two editors silently overwrite each other — and the same weakness is what API-ARX-TC-005 suspects in prescription review, which has no version at all.

PUT `/api/admin/content/pages` — AUTH: ADMIN

Static page bodies — terms, privacy, delivery, returns, about. Same `{ content, version }` shape. DESTRUCTIVE

PUT `/api/admin/content/faqs` — AUTH: ADMIN

FAQ list. Same shape. DESTRUCTIVE

POST `/api/admin/content/images` — AUTH: ADMIN

Multipart upload for editorial images, part `file`. Returns the URL to reference from content.

5.8 Settings

GET `/api/admin/settings` — AUTH: ADMIN

```
{ "details": {...}, "delivery": {...}, "payments": {...},
  "social": {...}, "districts": [ 64 names ] }
```

The writable mirror of `/api/store/config`. The four PUTs below each save one group.

PUT `/api/admin/settings/details` — AUTH: ADMIN

Name, hotline, e-mail, address. DESTRUCTIVE

PUT `/api/admin/settings/delivery` — AUTH: ADMIN

Charges, free-delivery threshold, home district, maximum quantity. **DESTRUCTIVE — HIGH BLAST RADIUS**

> Changing delivery settings alters the expected result of every pricing case in module API-CHK and every checkout case in the UI suite. API-AST-TC-004 is written to read the current values, change them, assert `/api/store/config` reflects the change, and then **restore the originals in a teardown that runs even on failure**. Without that restore a single failed run leaves the environment mispriced for everyone.

PUT `/api/admin/settings/payments` — AUTH: ADMIN

Payment methods and wallet numbers. **DESTRUCTIVE**

PUT `/api/admin/settings/social` — AUTH: ADMIN

Social links. **DESTRUCTIVE**, lowest risk of the four.

5.9 Reports

GET `/api/admin/reports/{key}` — AUTH: ADMIN

Parameters `from`, `to` (both `YYYY-MM-DD`), `group` and `status`. Six keys exist — each was probed directly:

KEY	STATUS	REPORT
`sales`	200	Sales over the period
`orders`	200	Orders, groupable by status
`products`	200	Product performance
`customers`	200	Customer activity
`prescriptions`	200	Prescription volume and decisions
`payments`	200	Payment status breakdown
`inventory`, `revenue`, `top-products`	404	Do not exist — useful negative fixtures

API-ARP-TC-006 asserts that `from` later than `to` is rejected rather than silently returning an empty report, and API-ARP-TC-008 that report totals reconcile with `/api/admin/orders` over the same window. A reporting figure that cannot be reconciled against the underlying records is worse than no figure at all.

GET `/api/admin/reports/{key}/export` AUTH: ADMIN

Same parameters plus `format`. Returns a file download named `north-pharmacy-{key}-{from}-to-{to}.{format}`.

API-ARP-TC-009 asserts the row count in the export matches the JSON report, and API-SEC-TC-020 that a field beginning `=`, `+`, `-` or `@` is neutralised — CSV formula injection, which matters because these exports are opened in Excel by people who trust them.

6 Test strategy

6.1 Objectives

1. **Prove the contract.** There is no OpenAPI document and no version segment in the path, so any field rename is instantly breaking and invisible until a screen misbehaves. Contract assertions are the backbone of the plan.
2. **Prove the business rules at the layer that owns them.** Delivery charges, the free-delivery threshold, quantity caps and prescription gating are enforced by the server. Testing them through a browser tests the browser as well, slowly.
3. **Prove authorisation on every object.** Customer against customer, customer against administrator, anonymous against everything. In a pharmacy this includes medical records, so it is a safety requirement and not only a security one.
4. **Settle questions the UI suite cannot answer.** Section 10 lists four defects in DEFECTS.md whose root cause or true severity is only determinable from this layer.
5. **Give the load plan a verified map.** NP-LT-001 depends on knowing which endpoints exist, what they cost and which are rate limited.

6.2 Test levels and what belongs where

LEVEL	BELONGS HERE	DOES NOT BELONG HERE
Contract	Status code, content type, field presence, field type, envelope shape	Exact values that move with seed data
Functional	Filtering, sorting, paging, pricing arithmetic, state transitions	Visual layout, copy, Bangla glyph shaping
Negative	Validation messages, boundary values, malformed input, wrong types	Browser-side validation, which the API never sees
Authorisation	401, 403 and 404 for anonymous, cross-customer and cross-role access	Password policy strength, which is a product decision
Integration	Agreement between endpoints: facets against totals, summary against lists, reports against orders	End-to-end journeys through the UI — NP-QA-TC-001 Part C owns those
Non-functional	Single-request latency, payload size, security headers, caching directives	Sustained load, concurrency, soak — NP-LT-001 owns those

6.3 Risk ranking

Priority is assigned by consequence of failure, not by how easy the case is to write.

PRI	MEANING	APPLIED TO
P1	Blocks release. Run on every build	Authentication, the CSRF guard, every authorisation boundary, prescription gating and review, money arithmetic, order integrity, upload validation

PRI	MEANING	APPLIED TO
P2	Run before every release	Catalogue filtering and paging, cart rules, account management, admin lists, reports, dashboard arithmetic
P3	Run on the full regression cycle	Content and settings reads, cosmetic fields, enumeration edge cases, payload-size checks

Two risks dominate and set the order of work. The first is **prescription handling**: upload validation (API-SEC-TC-012 to 016), access control (API-SEC-TC-005 to 008), review integrity (API-ARX-TC-005) and the Rx gate at checkout (API-CHK-TC-013). A pharmacy that dispenses against an unverified prescription has a regulatory problem, not a bug. The second is **money**: if `/api/cart/quote` and `/api/checkout` disagree, customers are charged something other than what they were shown.

6.4 Environment, data and isolation

Target	`BASE_URL`, default `https://202.4.101.118:1165`
TLS	`ignoreHTTPSErrors: true` — standing pre-condition NP-QA-TC-001 §2.5
Concurrency	`workers: 1`. One shared account per role; see 9.5
Seed data	Section 2.4. Reseed after any destructive run
Reads	Safe to run at any time against any environment
Writes	Gated. `RUN_DESTRUCTIVE=1`, disposable environment, reseed afterwards
Throttle cases	`RUN_THROTTLE=1`, run alone, never in a shared window

> **Three gates, inherited from the UI suite**
>
> `destructive()` — writes data the whole team shares. `manual()` — cannot be judged by a machine; at this layer only prescription *legibility* qualifies. `blocked()` — the feature is switched off, so the case cannot run; all three password-recovery cases are Blocked today. A gated case still appears in the report with its reason, so coverage stays honest.

6.5 Entry and exit criteria

ENTRY	EXIT
Environment reachable, both accounts valid	100% of P1 cases executed, zero failures
Seed data of section 2.4 present	≥ 95% of P2 cases executed, no open Critical or High defect
Suite runs green against the previous build	Every Blocked case carries a written reason and an owner
Destructive window agreed if write cases are in the run	Section 10 findings each resolved, accepted or scheduled
—	Any contract change is reflected in sections 4 and 5 before sign-off

6.6 Defect handling

An API failure is raised with the request line, the full request headers minus credentials, the request body, the response status, the response body and the `traceId`. The `traceId` is the single most useful field in the report: it lets a developer find the exact server-side trace without reproducing anything. Severity follows NP-QA-TC-001 §7.2, with one

addition — **any authorisation failure is Critical regardless of the data involved**, because the boundary itself is the control.

7 Test case catalogue

233 cases across 20 modules. Every case is one or a small number of HTTP requests with a machine-checkable expected result. Cases marked D are destructive and gated; B are blocked by a disabled feature; T trip a throttle and run alone.

MODULE	CASES	P1	COVERS
API-CNV	10	8	Cross-cutting conventions: CSRF guard, error shape, status codes
API-STO	6	1	Store config and content
API-CAT	26	4	Catalogue search, filter, sort, page, facets, product detail
API-AUT	23	14	Login, register, logout, session, recovery
API-CRT	16	6	Cart state and quantity rules
API-WSH	6	1	Wishlist
API-ACC	16	5	Account summary, profile, password, addresses
API-RXC	10	7	Customer prescriptions and file access
API-CHK	14	11	Quote arithmetic and order placement
API-ORD	13	7	Order list, detail, cancel, reorder
API-ADM	8	2	Admin summary and dashboard
API-AOR	10	4	Admin orders and status transitions
API-ARX	10	8	Admin prescriptions and review integrity
API-AIN	9	2	Admin inventory and product content
API-ACU	5	2	Admin customers
API-ACN	6	1	Admin content and optimistic concurrency
API-AST	8	2	Admin settings
API-ARP	9	1	Admin reports and exports
API-SEC	20	20	Authorisation, injection, upload abuse, headers
API-NFR	8	0	Latency, payload size, caching, headers
TOTAL	**233**	**106**	

7.1 API-CNV — conventions

ID	SCENARIO	PRI	EXPECTED
API-CNV-TC-001	Any GET with the CSRF header	P1	200, `application/json; charset=utf-8`
API-CNV-TC-002	GET without the CSRF header	P1	400 `{"title":"Request rejected.","status":400}`

ID	SCENARIO	PRI	EXPECTED
API-CNV-TC-003	POST `/api/auth/login` without the header	P1	400 Request rejected; no cookie issued
API-CNV-TC-004	PUT `/api/cart/items/{id}` without the header	P1	400 Request rejected; cart unchanged
API-CNV-TC-005	DELETE `/api/cart` without the header	P1	400 Request rejected; cart unchanged
API-CNV-TC-006	POST with header value other than `NorthPharmacy`	P1	400 Request rejected
API-CNV-TC-007	Validation error shape on a known-bad field	P1	`type`, `title`, `status`, `errors`, `traceId` all present
API-CNV-TC-008	404 problem document shape	P2	`errors` absent; `traceId` present
API-CNV-TC-009	Malformed JSON body on any POST	P1	400, never 500, and no stack trace in the body
API-CNV-TC-010	Wrong method on a known path, e.g. DELETE on `/api/store/config`	P3	405, and the body carries no framework detail

7.2 API-STO — store

ID	SCENARIO	PRI	EXPECTED
API-STO-TC-001	GET config returns all rule fields	P2	The nine §2.5 fields present with numeric types
API-STO-TC-002	Config values match §2.6 reference data	P2	60, 120, 2000, 200, Dhaka
API-STO-TC-003	`paymentMethods` contains COD	P1	At least one method; COD present
API-STO-TC-004	`departments` category keys resolve	P2	Every key exists in `/api/catalog/categories`
API-STO-TC-005	Content placeholders resolve against config	P2	Every `{token}` in content has a config key
API-STO-TC-006	Config is readable anonymously	P3	200 with no cookie

7.3 API-CAT — catalogue

ID	SCENARIO	PRI	EXPECTED
API-CAT-TC-001	Default product list	P2	200; envelope complete; 24 items
API-CAT-TC-002	Every item carries all 26 contract fields	P1	No field missing on any item in the page
API-CAT-TC-003	Field types are stable	P1	Prices numeric, flags boolean, ids GUID strings
API-CAT-TC-004	Search by brand `q=napa`	P2	33 results; title `Results for “napa”`
API-CAT-TC-005	Search by generic `q=Paracetamol`	P2	Non-empty; every item matches on generic or name
API-CAT-TC-006	Partial term `q=nap`	P2	Superset of the `napa` result
API-CAT-TC-007	No-result term `q=zzzqqqxyz`	P2	200, `total: 0`, `items: []` — not 404
API-CAT-TC-008	Bangla term `q=নাপা`	P2	200; UTF-8 echoed intact in `title`

ID	SCENARIO	PRI	EXPECTED
API-CAT-TC-009	Category counts reconcile	P2	`productCount` equals `total` for `?cat={key}`, all 11
API-CAT-TC-010	Filter `cat=tablets-capsules`	P2	7,539; every item has that `categoryKey`
API-CAT-TC-011	Filter `dept=medicines`	P2	11,561; title `Medicines`
API-CAT-TC-012	Filter `mfr={guid}`	P2	491 for Incepta; every item has that `manufacturerId`
API-CAT-TC-013	Filter `inStock=true`	P2	3,084; no item with `inStock: false`
API-CAT-TC-014	Filter `offers=true`	P2	2; every item `hasDiscount: true`
API-CAT-TC-015	Combined `cat` + `inStock`	P2	Total not greater than either filter alone
API-CAT-TC-016	Sort `price-asc`	P2	Prices non-decreasing across the page
API-CAT-TC-017	Sort `price-desc`	P2	Prices non-increasing
API-CAT-TC-018	Sort `name`	P3	Names in ascending collation order
API-CAT-TC-019	`pageSize` clamp	P2	1→12, 24→24, 500→60; echoed value matches item count
API-CAT-TC-020	`page` clamp	P2	`page=99999` returns the last page, not empty
API-CAT-TC-021	Unknown parameter `category=`	P3	Currently ignored; pinned so a future change is caught
API-CAT-TC-022	Unknown parameter `manufacturer=`	P3	Currently ignored; pinned
API-CAT-TC-023	Unknown parameter `genericId=`	P3	Currently ignored; pinned
API-CAT-TC-024	Paging is consistent — no duplicate id across pages 1 to 5	P2	Union size equals the sum of page sizes
API-CAT-TC-025	Product detail by GUID	P1	200; `product`, `description`, `generic`, `alternatives`, `related`
API-CAT-TC-026	Product detail by slug, unknown GUID, malformed id	P2	404 on all three

7.4 API-AUT — authentication

ID	SCENARIO	PRI	EXPECTED
API-AUT-TC-001	Valid customer login	P1	200; user object; `role: Customer`; cookie set
API-AUT-TC-002	Valid admin login	P1	200; `role: Admin`
API-AUT-TC-003	Cookie attributes	P1	`HttpOnly`, `Secure`, `SameSite=Lax`, `Path=/`
API-AUT-TC-004	`rememberMe: true`	P2	Expiry about 14 days ahead
API-AUT-TC-005	`rememberMe` omitted	P2	Session cookie, no expiry
API-AUT-TC-006	Wrong password	P1	400; “mobile number and password don’t match”; no cookie
API-AUT-TC-007	Unregistered number	P1	Identical message to TC-006 — no account enumeration

ID	SCENARIO	PRI	EXPECTED
API-AUT-TC-008	Missing password field	P2	400; `errors.password` = "Enter your password"
API-AUT-TC-009	Empty body	P2	400 with field errors, never 500
API-AUT-TC-010	Password is never echoed	P1	No request password substring in any response
API-AUT-TC-011	`/api/auth/me` anonymous	P2	200 `{"user": null}` — not 401
API-AUT-TC-012	Logout then reuse the cookie	P1	Logout 204; subsequent `/api/orders` 401
API-AUT-TC-013	Tampered cookie value	P1	401; not 500 and not accepted
API-AUT-TC-014	Cookie from a logged-out session	P1	401
API-AUT-TC-015	Register with a duplicate number	P2	400; `errors.phone` = already exists
API-AUT-TC-016	Register with an invalid number	P2	400; 01XXXXXXXXX message
API-AUT-TC-017	Register with a common password	P2	400; "too common and easy to guess"
API-AUT-TC-018	Register validation short-circuits	P3	Exactly one field key in `errors`
API-AUT-TC-019	Register rate limit T	P2	429 by about the tenth call in a minute
API-AUT-TC-020	Rate limit clears	P3	400 again after roughly 60 s
API-AUT-TC-021	Recovery channels B	P2	200 `{"channels": []}` — feature off
API-AUT-TC-022	`forgot/send` B	P1	Blocked — no channel exists
API-AUT-TC-023	`forgot/reset` B	P1	Blocked — no code can be issued

7.5 API-CRT — cart

ID	SCENARIO	PRI	EXPECTED
API-CRT-TC-001	GET cart anonymous	P1	401
API-CRT-TC-002	GET cart signed in	P2	200; `items` and `notices` both present
API-CRT-TC-003	PUT creates a line	P2	200; line present at the requested quantity
API-CRT-TC-004	PUT sets an absolute quantity	P2	Second PUT of 3 yields 3, not 4
API-CRT-TC-005	Quantity 0	P2	400 "Minimum 1 per product."
API-CRT-TC-006	Quantity −1	P2	400 "Minimum 1 per product."
API-CRT-TC-007	Quantity 9999	P1	400 "Maximum 200 per product."
API-CRT-TC-008	Quantity above per-product stock (Maxima, stock 1)	P1	Rejected or clamped with a notice; never silently accepted
API-CRT-TC-009	Quantity exactly 200	P2	Accepted — the boundary is inclusive
API-CRT-TC-010	Non-integer quantity	P3	400, never 500
API-CRT-TC-011	Unknown product GUID	P2	404 "This product is no longer available."

ID	SCENARIO	PRI	EXPECTED
API-CRT-TC-012	DELETE one line	P2	200; line gone, others untouched
API-CRT-TC-013	DELETE whole cart	P2	200 `{"items":[],"notices":[]}`
API-CRT-TC-014	`notices` always present	P1	Array on every cart response, empty if nothing to say
API-CRT-TC-015	Cart survives re-authentication	P2	Same contents after logout and login
API-CRT-TC-016	Merge on collision D	P1	Documented and stable: summed or replaced, never doubled unpredictably

7.6 API-WSH — wishlist

ID	SCENARIO	PRI	EXPECTED
API-WSH-TC-001	Anonymous access	P1	401 on all three endpoints
API-WSH-TC-002	Toggle adds	P2	200; id appears in `/api/wishlist/ids`
API-WSH-TC-003	Toggle twice returns to start	P2	Wishlist identical to the initial state
API-WSH-TC-004	Toggle an unknown GUID	P3	404, never 500
API-WSH-TC-005	`/wishlist` and `/wishlist/ids` agree	P2	Same set of ids, same length
API-WSH-TC-006	`wishlistCount` in account summary agrees	P3	Equals the id array length

7.7 API-ACC — account

ID	SCENARIO	PRI	EXPECTED
API-ACC-TC-001	Summary anonymous	P1	401
API-ACC-TC-002	Summary signed in	P2	All nine top-level keys present
API-ACC-TC-003	`memberSince` present and parseable	P2	Valid ISO-8601 in the past
API-ACC-TC-004	`orderCount` matches `/api/orders`	P2	Equal
API-ACC-TC-005	`lifetimeSpend` equals the sum of non-cancelled order totals	P2	Equal to two decimal places
API-ACC-TC-006	`recentOrders` is a prefix of the order list	P3	Same ids, same order, newest first
API-ACC-TC-007	No sensitive field in the payload	P1	No hash, salt, token or internal key
API-ACC-TC-008	Update profile D	P2	200; summary reflects the change
API-ACC-TC-009	Profile with an invalid e-mail D	P2	400 with a field error
API-ACC-TC-010	Change password with the wrong current one D	P1	400; password unchanged
API-ACC-TC-011	Add an address with an unknown district D	P2	400 — must be one of the 64

ID	SCENARIO	PRI	EXPECTED
API-ACC-TC-012	Add a valid address D	P2	200; appears in the list and in checkout options
API-ACC-TC-013	Add an address with a malformed phone D	P2	400
API-ACC-TC-014	Set default D	P2	Exactly one address flagged default afterwards
API-ACC-TC-015	Delete an address D	P2	Removed from the list
API-ACC-TC-016	Delete another customer's address D	P1	404; the address still exists

7.8 API-RXC — prescriptions, customer side

ID	SCENARIO	PRI	EXPECTED
API-RXC-TC-001	List anonymous	P1	401
API-RXC-TC-002	List returns only the caller's uploads	P1	Every id belongs to the signed-in customer
API-RXC-TC-003	Upload a valid JPG D	P1	200; record created with status Pending
API-RXC-TC-004	Upload a valid PDF D	P2	200; `isImage: false`
API-RXC-TC-005	Upload with no `file` part D	P2	400
API-RXC-TC-006	`patientName` and `note` stored D	P3	Echoed back on the record
API-RXC-TC-007	Own file retrieval D	P1	200 with the original content type
API-RXC-TC-008	Another customer's file	P1	404 — verified against prescription 1006
API-RXC-TC-009	File anonymous	P1	401
API-RXC-TC-010	Prescription legibility manual	P2	A person confirms the stored scan is readable

7.9 API-CHK — quote and checkout

ID	SCENARIO	PRI	EXPECTED
API-CHK-TC-001	Quote, one item, Dhaka	P1	Delivery 60; total = subtotal + 60
API-CHK-TC-002	Quote, Chittagong	P1	Delivery 120
API-CHK-TC-003	Quote, Sylhet and Rangpur	P2	Delivery 120 for both
API-CHK-TC-004	Subtotal just below 2,000	P1	Delivery charged in full
API-CHK-TC-005	Subtotal exactly 2,000	P1	Free delivery — the boundary is inclusive
API-CHK-TC-006	Subtotal above 2,000	P1	Delivery 0
API-CHK-TC-007	Free delivery applies outside Dhaka too	P1	Delivery 0 at 2,000 in Chittagong
API-CHK-TC-008	Quote arithmetic	P1	subtotal − discount + delivery = total, to 2 dp
API-CHK-TC-009	Quote with an unknown district	P2	400 rather than a default charge

ID	SCENARIO	PRI	EXPECTED
API-CHK-TC-010	Quote with an empty basket	P3	200 with zero totals, or 400 — pinned either way
API-CHK-TC-011	Quote matches the order actually placed D	P1	Every money field identical
API-CHK-TC-012	Checkout options complete	P2	addresses, prescriptions, 64 districts, payment methods
API-CHK-TC-013	Rx product without `prescriptionId` D	P1	**Rejected.** See finding F-4 — order 104027 suggests it is not
API-CHK-TC-014	Checkout with neither `addressId` nor inline address D	P1	400

7.10 API-ORD — orders

ID	SCENARIO	PRI	EXPECTED
API-ORD-TC-001	List anonymous	P1	401
API-ORD-TC-002	List returns only the caller's orders	P1	Every order belongs to the signed-in customer
API-ORD-TC-003	List is newest first	P3	`createdOn` non-increasing
API-ORD-TC-004	List is unpaginated	P3	Bare array; no envelope. Pinned — see F-5
API-ORD-TC-005	Detail contract	P2	All 24 fields including `items` and `history`
API-ORD-TC-006	Money reconciles	P1	subtotal − discount + delivery = total, every order
API-ORD-TC-007	Line totals reconcile	P1	unitPrice × quantity = lineTotal; sum = subtotal
API-ORD-TC-008	`pieceCount` and `productCount`	P2	Sum of quantities, and number of lines
API-ORD-TC-009	Cancel when `canCancel: false` D	P1	Refused; status unchanged
API-ORD-TC-010	Cancel when `canCancel: true` D	P2	200; status Cancelled; history appended
API-ORD-TC-011	Cancel another customer's order D	P1	404; that order unchanged
API-ORD-TC-012	Detail for another customer's order	P1	404 — verified on orders 1 and 104000
API-ORD-TC-013	Reorder with an out-of-stock line D	P2	Surfaced in `notices`; call still succeeds

7.11 API-ADM, API-AOR, API-ARX — administration

ID	SCENARIO	PRI	EXPECTED
API-ADM-TC-001	Summary anonymous	P1	401
API-ADM-TC-002	Summary as customer	P1	403 — verified
API-ADM-TC-003	Badge counts reconcile	P2	All four match the matching list `counts`
API-ADM-TC-004	Dashboard contract	P2	All 17 keys present
API-ADM-TC-005	`days=7`, `30`, `90`	P2	Accepted; `days` echoed

ID	SCENARIO	PRI	EXPECTED
API-ADM-TC-006	`from` / `to` span `days`	P2	Difference equals `days`
API-ADM-TC-007	`daily` length	P2	Exactly `days` entries, no gaps
API-ADM-TC-008	Dates use Asia/Dhaka	P3	Boundaries agree with Dhaka local time, not UTC
API-AOR-TC-001	Order list envelope	P2	`items`, `total`, `page`, `totalPages`, `counts`
API-AOR-TC-002	`counts` sums to `total` with no filter	P2	21+6+3+3+4+13 = 50
API-AOR-TC-003	Filter by each of the six statuses	P2	Every item carries that status; total matches `counts`
API-AOR-TC-004	Admin detail matches customer detail	P2	Shared fields identical
API-AOR-TC-005	Search `q` by order number and phone	P2	Both find the known order
API-AOR-TC-006	`status=active`	P3	Returns in-progress orders only
API-AOR-TC-007	Valid transition Pending→Confirmed D	P1	200; status changed
API-AOR-TC-008	Invalid transition Delivered→Pending D	P1	Refused
API-AOR-TC-009	Every change appends to `history` D	P1	New entry with status, note and timestamp
API-AOR-TC-010	Payment status change D	P2	200; reflected on both admin and customer views
API-ARX-TC-001	List as customer	P1	403 — verified
API-ARX-TC-002	List contract	P2	All 15 fields including `fileUrl` and `orderIds`
API-ARX-TC-003	Filter by status	P2	Pending, Approved, Rejected, all behave correctly
API-ARX-TC-004	Approve a Pending prescription D	P1	200; `reviewedBy` and `reviewedOn` set
API-ARX-TC-005	**Reject an already-Approved prescription** D	P1	**Refused (409 or 400).** Decides the severity of DEFECTS.md issue 6
API-ARX-TC-006	Approve an already-Rejected prescription D	P1	Refused
API-ARX-TC-007	Review fields are never cleared D	P1	`reviewedBy` / `reviewedOn` persist
API-ARX-TC-008	Invalid status value D	P2	400; record unchanged
API-ARX-TC-009	Two concurrent reviews D	P1	One wins; the other is refused. Mirrors NP-RXA-TC-017
API-ARX-TC-010	Delete a prescription attached to an order D	P1	Refused, or the order reference is handled — never dangling

7.12 API-AIN, API-ACU, API-ACN, API-AST, API-ARP

ID	SCENARIO	PRI	EXPECTED
API-AIN-TC-001	Inventory as customer	P1	403
API-AIN-TC-002	Each `view` returns its `counts` figure	P2	low 772, out 144, nophoto 3067, all 17453
API-AIN-TC-003	`view=low` items really are low	P2	Every item `stock` ≤ 10

ID	SCENARIO	PRI	EXPECTED
API-AIN-TC-004	**Admin search matches catalogue search**	P2	Counts agree for Napa, Dps, Paracetamol. *Currently fails — DEFECTS.md issue 2*
API-AIN-TC-005	Filter by `outlet`	P3	Accepted; results scoped to that outlet
API-AIN-TC-006	`outlets` list present	P3	Three outlets with id and name
API-AIN-TC-007	Product content read	P3	200 with description, discount, visibility
API-AIN-TC-008	Discount outside 0–100 D	P1	400; price unchanged
API-AIN-TC-009	Hiding a product removes it from the catalogue D	P2	Absent from `/api/catalog/products`
API-ACU-TC-001	Customers as customer role	P1	403
API-ACU-TC-002	List envelope and paging	P2	`items`, `total`, `page`, `totalPages`
API-ACU-TC-003	Search by name and by mobile	P2	Both find the seeded customer
API-ACU-TC-004	Figures match the customer's own summary	P2	Order count and lifetime spend agree
API-ACU-TC-005	No credential material in the payload	P1	No hash, salt or token field
API-ACN-TC-001	Content read as admin	P3	home, pages, faqs, icons
API-ACN-TC-002	Content as customer	P1	403
API-ACN-TC-003	Save home content D	P2	200; `/api/store/content` reflects it
API-ACN-TC-004	**Stale `version` is rejected** D	P2	Second save with the old version refused — no lost update
API-ACN-TC-005	Save pages and FAQs D	P3	200; reflected in store content
API-ACN-TC-006	Content image upload D	P3	200; URL returned and reachable
API-AST-TC-001	Settings as customer	P1	403
API-AST-TC-002	Settings contract	P2	details, delivery, payments, social, districts
API-AST-TC-003	Districts list is the catalogue list	P3	Same 64 names
API-AST-TC-004	Change delivery charge, verify, restore D	P1	Config reflects it; teardown restores on failure too
API-AST-TC-005	Settings and store config agree — charges	P2	Identical values
API-AST-TC-006	— threshold and max quantity	P2	Identical values
API-AST-TC-007	— payment methods	P2	Identical list
API-AST-TC-008	Negative delivery charge D	P2	400; settings unchanged
API-ARP-TC-001	Reports as customer	P1	403
API-ARP-TC-002	All six valid keys	P2	200 for sales, orders, products, customers, prescriptions, payments
API-ARP-TC-003	Unknown key	P3	404 for inventory, revenue, top-products

ID	SCENARIO	PRI	EXPECTED
API-ARP-TC-004	Date range is honoured	P2	Nothing outside `from` .. `to`
API-ARP-TC-005	Single-day range	P3	200; that day only
API-ARP-TC-006	`from` later than `to`	P2	400 — not a silent empty report
API-ARP-TC-007	Malformed date	P3	400, never 500
API-ARP-TC-008	Sales reconcile with orders	P2	Report total equals the sum over the same window
API-ARP-TC-009	Export row count matches the JSON report	P2	Equal; filename follows the documented pattern

7.13 API-NFR — non-functional single-request checks

These are per-request checks, not load tests. Sustained behaviour belongs to NP-LT-001.

ID	SCENARIO	PRI	EXPECTED
API-NFR-TC-001	Warm single-request latency, every read endpoint	P3	Within 3× the section 10 baseline
API-NFR-TC-002	Catalogue page payload size	P3	≤ 25 KB at `pageSize=24`
API-NFR-TC-003	Security headers on success	P3	HSTS, nosniff, DENY, Referrer-Policy all present
API-NFR-TC-004	Security headers on error	P3	Same four present on a 400 and a 404
API-NFR-TC-005	Authenticated responses are not cacheable	P3	`no-cache,no-store` on every authenticated read
API-NFR-TC-006	No server version disclosure	P3	`Server` and `X-Powered-By` carry no version
API-NFR-TC-007	No stack trace on any error	P3	No file path or frame in any 4xx or 5xx body
API-NFR-TC-008	Compression offered when requested	P3	*Currently not negotiated — finding F-8*

8 Security and authorisation tests

Twenty cases, every one P1. These are separated from the functional catalogue because they are run as a block before any release and because a failure here is Critical by definition — the boundary itself is the control, so its strength does not depend on how interesting the data behind it is.

Eight of the twenty have already been executed during this survey and **passed**. They are marked below. That is a genuinely good result for a product handling medical records, and the point of keeping them in the suite is to notice the day it stops being true.

8.1 Authentication and session

ID	SCENARIO	STATUS	EXPECTED
API-SEC-TC-001	Every protected endpoint without a cookie	PASSED	401, empty body, no data leak
API-SEC-TC-002	Session cookie flags	PASSED	HttpOnly, Secure, SameSite=Lax
API-SEC-TC-003	Login does not reveal whether an account exists	PASSED	Identical message for wrong password and unknown number
API-SEC-TC-004	Session is invalidated by logout	To run	Replayed cookie returns 401

8.2 Object-level authorisation — the pharmacy boundary

ID	SCENARIO	STATUS	EXPECTED
API-SEC-TC-005	Customer reads another customer's prescription file	PASSED	404 — tested against prescription 1006
API-SEC-TC-006	Anonymous reads a prescription file	PASSED	401
API-SEC-TC-007	Administrator reads any prescription file	PASSED	200 with the correct content type
API-SEC-TC-008	Customer reads another customer's order	PASSED	404 — tested on orders 1 and 104000
API-SEC-TC-009	Customer calls every `/api/admin/*` route	PASSED (4 of 31)	403 on all 31; extend the sample to the full set
API-SEC-TC-010	Customer calls the prescription review action directly	PASSED	403 — the UI control is not the only guard
API-SEC-TC-011	Order and prescription ids are not guessable in sequence	To run	Walking ids yields 404, never another customer's record

8.3 Upload abuse — the highest open risk

> **Server-side upload validation is unproven**
>
> The accepted extensions and the 8 MB limit for prescriptions, and the 3 MB limit for images, are enforced in the browser. Nothing yet demonstrates the server repeats those checks, and a direct POST bypasses the client entirely. The suite already carries the fixtures — `oversized-9mb.jpg`, `payload.exe`, `renamed-exe.jpg` and `not-a-prescription.txt` — and these five cases post each one directly. Run them before the next release.

ID	SCENARIO	FIXTURE	EXPECTED
API-SEC-TC-012	Prescription upload above the size limit	`oversized-9mb.jpg`	413 or 400; never stored
API-SEC-TC-013	Executable uploaded as a prescription	`payload.exe`	400 on content type; never stored
API-SEC-TC-014	Executable renamed with an image extension	`renamed-exe.jpg`	Rejected on *content*, not on extension
API-SEC-TC-015	Text file uploaded as a prescription	`not-a-prescription.txt`	400
API-SEC-TC-016	Path traversal in the upload filename	`../../web.config`	Sanitised; nothing written outside the upload root
API-SEC-TC-017	Product image above 3 MB	generated	413 or 400
API-SEC-TC-018	SVG containing a script element as a product image	generated	Rejected, or served with a non-renderable content type and `Content-Disposition: attachment`

8.4 Injection and output safety

ID	SCENARIO	STATUS	EXPECTED
API-SEC-TC-019	Newsletter subscription does not leak membership	To run	Identical response for a new and an existing address
API-SEC-TC-020	CSV formula injection in report exports	To run	Leading `=`, `+`, `-`, `@` neutralised

SQL injection was probed opportunistically during the survey: `q=' OR 1=1--` against `/api/catalog/suggest` returned `[]` with 200, which is the correct result for a parameterised query — the string was treated as a search term and matched nothing. The same payload belongs in every text parameter the catalogue accepts, and that sweep is folded into API-CAT rather than duplicated here.

Stored cross-site scripting is *not* an API concern at this layer and is deliberately excluded: the API returning `<script>alert(1)</script>` as a product name is correct behaviour, because escaping is the renderer's responsibility. The UI suite owns that through NP-SRC-TC-009. The one exception is the SVG case above, where the file is served directly by the server rather than rendered as data by the client.

9 Automation

The suite extends the existing Playwright project rather than introducing a second tool. Playwright runs API tests without a browser through `request`, so the API cases share the configuration, the reporter, the CI workflow and the gating helpers that the 434 UI cases already use — and a single run reports both layers together.

9.1 Structure

```
tests/api/
  00-conventions.api.spec.ts      API-CNV   10 cases
  01-store.api.spec.ts            API-STO    6
  02-catalog.api.spec.ts          API-CAT   26
  03-auth.api.spec.ts             API-AUT   23
  04-cart.api.spec.ts             API-CRT   16
  05-wishlist.api.spec.ts         API-WSH    6
  06-account.api.spec.ts          API-ACC   16
  07-prescriptions.api.spec.ts    API-RXC   10
  08-checkout.api.spec.ts         API-CHK   14
  09-orders.api.spec.ts           API-ORD   13
  10-admin-dashboard.api.spec.ts  API-ADM    8
  11-admin-orders.api.spec.ts     API-AOR   10
  12-admin-rx.api.spec.ts         API-ARX   10
  13-admin-inventory.api.spec.ts  API-AIN    9
  14-admin-customers.api.spec.ts  API-ACU    5
  15-admin-content.api.spec.ts    API-ACN    6
  16-admin-settings.api.spec.ts   API-AST    8
  17-admin-reports.api.spec.ts    API-ARP    9
  18-security.api.spec.ts         API-SEC   20
  19-nfr.api.spec.ts              API-NFR    8

utils/apiClient.ts                extended — see 9.2
fixtures/api-fixtures.ts          anonApi / customerApi / adminApi
```

9.2 The client must set the CSRF header centrally

The existing `utils/apiClient.ts` is a thin wrapper that does not send `X-Requested-With`, so every write through it currently returns `400 Request rejected`. One change fixes the whole suite:

```
import { APIRequestContext } from '@playwright/test';

/** Required on every request — see NP-API-TS-001 §3.3. */
const GUARD = { 'X-Requested-With': 'NorthPharmacy', Accept: 'application/json' };

export class ApiClient {
  constructor(private request: APIRequestContext, private baseUrl: string) {}

  private h(extra: Record<string, string> = {}) { return { ...GUARD, ...extra }; }

  get(path: string, headers = {})            { return this.request.get(this.baseUrl + path,    {
headers: this.h(headers) }); }
  post(path: string, data?: unknown, h = {}) { return this.request.post(this.baseUrl + path,   { data,
headers: this.h(h) }); }
  put(path: string, data?: unknown, h = {})  { return this.request.put(this.baseUrl + path,    { data,
headers: this.h(h) }); }
  delete(path: string, headers = {})         { return this.request.delete(this.baseUrl + path, {
headers: this.h(headers) }); }

  /** For API-CNV-TC-002..006 only: deliberately omit the guard. */
  unguarded(method: 'get' | 'post', path: string, data?: unknown) {
    return this.request.fetch(this.baseUrl + path, { method, data });
  }
}
```

The `unguarded` escape hatch exists so the CSRF cases can prove the guard works. Without it they would have to bypass the client and the assertion would drift from what production code actually sends.

9.3 Fixtures

Three request contexts, each signed in once per worker and reused, mirroring how `customerTest` and `adminTest` work for the UI suite.

```
// fixtures/api-fixtures.ts
import { test as base, request, APIRequestContext } from '@playwright/test';
import { ApiClient } from '../utils/apiClient';
import { BASE_URL, users } from '../data/testData';

async function signedIn(phone: string, password: string): Promise<APIRequestContext> {
  const ctx = await request.newContext({ baseURL: BASE_URL, ignoreHTTPSErrors: true });
  const res = await ctx.post('/api/auth/login', {
    headers: { 'X-Requested-With': 'NorthPharmacy' },
    data: { phone, password, rememberMe: true },
  });
  if (!res.ok()) throw new Error(`Login failed for ${phone}: ${res.status()} ${await res.text()}`);
  return ctx;                        // the np_web_auth cookie now rides on this context
}

export const test = base.extend<{
  anonApi: ApiClient; customerApi: ApiClient; adminApi: ApiClient;
}>({
  anonApi: async ({ playwright }, use) => {
    const ctx = await playwright.request.newContext({ ignoreHTTPSErrors: true });
    await use(new ApiClient(ctx, BASE_URL));
    await ctx.dispose();
  },
  customerApi: async ({}, use) => {
    const ctx = await signedIn(users.customer.mobile, users.customer.password);
    await use(new ApiClient(ctx, BASE_URL));
    await ctx.dispose();
  },
  adminApi: async ({}, use) => {
    const ctx = await signedIn(users.admin.mobile, users.admin.password);
    await use(new ApiClient(ctx, BASE_URL));
    await ctx.dispose();
  },
});
export { expect } from '@playwright/test';
```

9.4 Worked examples

A contract case — API-CAT-TC-002

```
const PRODUCT_FIELDS = [
  'id','name','slug','generic','genericId','formation','strength','subtitle','packUnit',
  'manufacturer','manufacturerId','categoryKey','categoryName','tone','art','mrp','price',
  'discountPercent','hasDiscount','inStock','lowStock','stockLeft','maxQty',
  'requiresPrescription','isFeatured','imageUrl',
] as const;

test('API-CAT-TC-002 | every list item carries the full product contract',
  { tag: ['@api', '@p1', '@cat'] }, async ({ anonApi }) => {
    const res = await anonApi.get('/api/catalog/products?page=1&pageSize=24');
    expect(res.status()).toBe(200);

    const body = await res.json();
    expect(body.items.length).toBeGreaterThan(0);
    for (const item of body.items) {
      for (const f of PRODUCT_FIELDS) {
        expect(item, `product ${item.id ?? '?'} is missing "${f}"`).toHaveProperty(f);
      }
      expect(typeof item.price).toBe('number');
      expect(typeof item.inStock).toBe('boolean');
    }
  });
```

A business-rule case — API-CHK-TC-005, the free-delivery boundary

```
test('API-CHK-TC-005 | delivery is free at exactly the threshold',
  { tag: ['@api', '@p1', '@chk'] }, async ({ anonApi }) => {
    // Read the rule rather than hard-coding it, so a settings change reads as config drift.
    const cfg = await (await anonApi.get('/api/store/config')).json();
    const { freeDeliveryThreshold, homeDistrict, deliveryChargeInside } = cfg;

    const product = { id: '4b588f69-008b-4e62-b98c-3f85c67199b5', price: 1.20 };
    const qty = Math.ceil(freeDeliveryThreshold / product.price);   // just reaches the threshold

    const quote = await (await anonApi.post('/api/cart/quote', {
      items: [{ productId: product.id, quantity: qty }], district: homeDistrict,
    })).json();

    expect(quote.subtotal).toBeGreaterThanOrEqual(freeDeliveryThreshold);
    expect(quote.deliveryCharge).toBe(0);
    expect(quote.total).toBeCloseTo(quote.subtotal - quote.discount, 2);

    // and one unit fewer must still be charged
    const below = await (await anonApi.post('/api/cart/quote', {
      items: [{ productId: product.id, quantity: qty - 1 }], district: homeDistrict,
    })).json();
    expect(below.deliveryCharge).toBe(deliveryChargeInside);
  });
```

An authorisation case — API-SEC-TC-005

```
test('API-SEC-TC-005 | a customer cannot read another customer's prescription file',
  { tag: ['@api', '@p1', '@sec'] }, async ({ customerApi, adminApi, anonApi }) => {
    const OTHER_RX = 1006;                    // belongs to a different customer

    expect((await anonApi.get(`/api/prescriptions/${OTHER_RX}/file`)).status()).toBe(401);
    expect((await customerApi.get(`/api/prescriptions/${OTHER_RX}/file`)).status()).toBe(404);

    const asAdmin = await adminApi.get(`/api/prescriptions/${OTHER_RX}/file`);
    expect(asAdmin.status()).toBe(200);       // the admin may read it
    expect(asAdmin.headers()['content-type']).toContain('application/pdf');
  });
```

A gated case — API-ARX-TC-005

```
destructive('API-ARX-TC-005 | an already-approved prescription cannot be rejected',
  { tag: ['@api', '@p1', '@arx'] }, async ({ adminApi }) => {
    const list = await (await adminApi.get('/api/admin/prescriptions?status=Approved')).json();
    test.skip(list.items.length === 0, 'No approved prescription to exercise.');
    const rx = list.items[0];

    const res = await adminApi.post(`/api/admin/prescriptions/${rx.id}/review`,
                                    { status: 'Rejected', note: 'API-ARX-TC-005' });

    // The pharmacist sign-off is an audit record. A second administrator must not overwrite it.
    expect(res.status(), 'a reviewed prescription was re-decided').toBeGreaterThanOrEqual(400);

    const after = await (await adminApi.get(`/api/admin/prescriptions?status=all`)).json();
    const same = after.items.find((p: any) => p.id === rx.id);
    expect(same.status).toBe('Approved');
    expect(same.reviewedBy).toBe(rx.reviewedBy);
    expect(same.reviewedOn).toBe(rx.reviewedOn);
  });
```

9.5 Running

```
npm run test:api                              # the whole API suite
npx playwright test --grep "@api and @p1"     # release gate
npx playwright test --grep "@api and @sec"    # the security block
npx playwright test --grep @cat               # one module

RUN_DESTRUCTIVE=1 npx playwright test --grep "@api and @arx"   # disposable env only
RUN_THROTTLE=1    npx playwright test --grep "@api and @throttle"
```

Add to `package.json`:

```
"test:api:p1":  "playwright test tests/api --grep @p1 --project=chromium",
"test:api:sec": "playwright test tests/api --grep @sec --project=chromium"
```

Parallelism — the one change worth making

The UI suite runs `workers: 1` because a browser session, a cart and a wishlist are all shared. The read-only API cases — roughly 140 of the 233 — have no such constraint and would run in parallel today with no change to the environment. The remaining cases need their own account each. Splitting the suite into a parallel read project and a serial write project is a small change that would take the API run from minutes to seconds; it is recommended but not required, and it is not assumed anywhere in this document.

9.6 Continuous integration

The existing `.github/workflows/playwright.yml` runs `npx playwright test`, which will pick up `tests/api` automatically. Two additions are recommended: run the API suite as a separate, earlier job, because it finishes in a fraction of the time and fails fast on a broken contract; and make the P1 API block a required check, since a broken authorisation boundary should never reach a review queue.

```
api:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: lts/* }
    - run: npm ci
    - run: npx playwright test tests/api --grep @p1 --project=chromium
      env:
        BASE_URL:       ${{ secrets.BASE_URL }}
        TEST_USER:      ${{ secrets.TEST_USER }}
        TEST_PASSWORD:  ${{ secrets.TEST_PASSWORD }}
        ADMIN_USER:     ${{ secrets.ADMIN_USER }}
        ADMIN_PASSWORD: ${{ secrets.ADMIN_PASSWORD }}
```

10 Findings from this survey

Eight observations made while mapping the API. None was found by running the test catalogue — the catalogue does not exist yet — so all eight are the by-product of looking at the system from a layer the existing suite does not reach. Four of them change the reading of defects already open in `DEFECTS.md`.

F-1 The product detail 502 is a delivery fault, not a catalogue fault P1

Relates to	DEFECTS.md issue 1 — Critical, P1, 21 cases in module A4 plus NP-SRC-TC-004 and NP-CNT-TC-008
Observed	`GET /api/catalog/products/4b588f69-...` returns **200** with complete data — product, description, generic, 8 alternatives, 10 related — in 3 ms, 12.1 KB
Meaning	The catalogue service and its data are healthy. The 502 is produced while serving the `/p/:id/:slug` *page*
Recommendation	Point the investigation at page delivery — server-side rendering, the IIS route or the proxy — not at the catalogue. 21 blocked UI cases depend on it

Also worth noting: the API accepts the GUID but not the slug, while the page URL contains both. If the page resolves by slug, that mismatch is itself a candidate cause.

F-2 `memberSince` exists in the API but not on the account page P3

Relates to	DEFECTS.md environment note — NP-ACC-TC-013 expects a "Member since" date
Observed	`GET /api/account/summary` returns `"memberSince": "2026-10-24T10:31:23.329Z"`
Meaning	Not missing data — a presentation gap. A client-side change, not a server one
Recommendation	Render it, then move NP-ACC-TC-013 from flagged to passing

F-3 Unknown query parameters are silently ignored P2

Observed	`?category=tablets-capsules` and `?manufacturer=<guid>` both return the full unfiltered 17,453 with HTTP 200. The real names are `cat` and `mfr`
Meaning	A misspelled filter produces a successful response containing the wrong data. There is no feedback of any kind — no warning field, no 400
Impact	Mostly an integration hazard rather than a user-facing one, but a dangerous class: a future client, an integration partner or a test can silently assert against unfiltered data and pass
Recommendation	Either reject unknown parameters with 400, or echo the applied filters in the response so a caller can verify them. Until then, cases API-CAT-TC-021 to 023 pin the current behaviour

F-4 A prescription-required order exists with no prescription attached P1

Observed	Order **NP-104027**: admin view `"hasRx": true, "prescriptionStatus": null`; customer view `"prescription": null`; its single line is Dps, `requiresPrescription: true`. The order sits in Pending
Meaning	Either the Rx gate at checkout does not require a prescription for a prescription-only product, or the prescription reference was lost after placement. Both are serious; the seeded data cannot tell them apart

Impact	For a pharmacy this is the control that makes dispensing lawful. The administration screen shows the order as ordinary work in the queue
Recommendation	Run API-CHK-TC-013 against a disposable environment — place an order for a prescription-only product with `prescriptionId: null` and record the status code. That single request distinguishes the two causes

> This is the one finding in this list that could not be settled without a destructive call, and it is also the one with the highest consequence. It should be scheduled into the next disposable-environment window rather than waiting for a full regression cycle.

F-5 `GET /api/orders` is unbounded P2

Observed	Returns a bare array of every order the customer has ever placed. No envelope, no `page`, no limit — the only collection in the API without paging
Impact	Payload and server cost grow without bound for the customers who matter most. It is also a contract inconsistency: a client written against any other collection will look for `items` and find an array
Recommendation	Add the standard envelope with a default page size, keeping the first page compatible for existing clients. Exercised under load by NP-LT-001 §6.4

F-6 429 carries no `Retry-After` P3

Observed	`POST /api/auth/register` returns 429 from about the tenth call in a minute, with no `Retry-After` and no `RateLimit-*` headers
Impact	A client cannot know how long to wait. The SPA hard-codes “wait a minute”, which will be wrong the moment the policy changes
Recommendation	Emit `Retry-After`. It is one header and it makes the limit self-describing

F-7 No Content-Security-Policy P2

Observed	HSTS, `nosniff`, `X-Frame-Options: DENY` and `Referrer-Policy` are all present and correct. `Content-Security-Policy` is absent. `X-Powered-By` is sent twice, both empty
Impact	The header set is otherwise good, which makes CSP the obvious remaining gap — and the SPA renders user-supplied strings such as product names and prescription notes
Recommendation	Add a CSP in report-only mode first, review the reports, then enforce. Remove the empty `X-Powered-By` pair while in that configuration

F-8 Responses are not compressed P2

Observed	No content encoding is negotiated. A catalogue page is 17.3 KB of JSON on the wire; `/api/catalog/home` is 19.3 KB
Impact	Negligible on the LAN this environment sits on, material on mobile data — which the test plan names as the dominant channel. JSON of this shape typically compresses by 80–85%
Recommendation	Enable Brotli or gzip response compression in IIS. Likely the single cheapest performance improvement available, and NP-LT-001 §9 quantifies it

10.9 Measured baseline

Warm single-request latency, 12 samples per endpoint, taken sequentially from a client on the same network. These are the figures API-NFR-TC-001 compares against, and the input to NP-LT-001.

ENDPOINT	P50	P95	SIZE	NOTE
`/api/catalog/districts`	1 ms	1 ms	0.7 KB	Cheapest — health-check target
`/api/auth/me`	1 ms	1 ms	<0.1 KB	Session probe
`/api/store/config`	2 ms	3 ms	0.9 KB	
`/api/catalog/categories`	2 ms	2 ms	1.8 KB	
`/api/catalog/manufacturers`	2 ms	2 ms	10.3 KB	
`/api/store/content`	3 ms	3 ms	7.7 KB	
`/api/catalog/products/{id}`	3 ms	4 ms	12.1 KB	Healthy — see F-1
`/api/catalog/suggest?q=nap`	5 ms	7 ms	2.0 KB	Fires on every keystroke
`/api/catalog/home`	5 ms	6 ms	19.3 KB	Largest payload
`/api/catalog/products?q=napa`	7 ms	13 ms	16.3 KB	
`/api/catalog/products`	15 ms	17 ms	17.3 KB	**Slowest and heaviest**

Every endpoint responds in single-digit or low double-digit milliseconds when not contended. The unfiltered catalogue query is five to fifteen times the cost of anything else, which is why NP-LT-001 weights it most heavily in the workload model.

11 Traceability and coverage

11.1 API modules against the UI specification

The API suite does not replace NP-QA-TC-001; it covers the server-side half of the same requirements, faster and more precisely. Where a UI module is listed as "partly replaced", the API case proves the rule and the UI case need only prove that the screen honours it.

UI MODULE	API MODULE	RELATIONSHIP	NOTE
A2 Catalogue / A3 Search	API-CAT	Partly replaces	Filtering, sorting and paging move to the API; the UI keeps rendering and interaction
A4 Product detail	API-CAT-TC-025/026	Diagnoses	Proves the data layer is sound while the page 502s
A5 Cart	API-CRT	Partly replaces	Quantity rules at the server; the UI keeps the stepper and totals display
A6 Checkout	API-CHK	Strengthens	Money boundaries become non-destructive through `/cart/quote`
A7 Login / A8 Register	API-AUT	Partly replaces	Validation and session semantics at the API
A9 Forgot password	API-AUT-TC-021..023	Mirrors	Blocked in both layers; the API shows *why* — `channels: []`
A10 Account	API-ACC	Partly replaces	Reveals `memberSince` — F-2
A12 Prescriptions	API-RXC, API-SEC	Strengthens	Adds upload abuse and access-control cases the browser cannot reach
B1 Admin orders	API-AOR	Partly replaces	Transition rules and audit history at the API
B3 Admin prescriptions	API-ARX	Resolves	API-ARX-TC-005 decides the severity of DEFECTS.md issue 6
B4 Admin inventory	API-AIN	Diagnoses	API-AIN-TC-004 localises the brand-search defect to query or screen
B8 Reports	API-ARP	Strengthens	Adds reconciliation against the underlying orders
D2 Security	API-SEC	Replaces	Authorisation is an API property; a browser can only sample it
D1 Compatibility / D3 Responsive / D4 Accessibility	—	No overlap	Presentation concerns; stay entirely in the UI suite

11.2 Endpoint coverage

GROUP	ENDPOINTS	COVERED	EXERCISED	GAP
Store	2	2	2	—
Catalogue	7	7	7	—

GROUP	ENDPOINTS	COVERED	EXERCISED	GAP
Authentication	7	7	4	3 blocked — recovery disabled
Cart	6	6	5	`merge` not exercised
Wishlist	3	3	3	—
Account	7	7	2	5 destructive
Prescriptions	3	3	2	Upload destructive
Checkout and orders	6	6	4	Checkout and cancel destructive
Newsletter	1	1	0	Creates a subscription
Admin read	13	13	11	—
Admin write	18	18	0	All destructive
TOTAL	**73**	**73**	**40**	**100% SPECIFIED, 55% EXERCISED**

Covered means a test case exists for it in section 7. **Exercised** means it was actually called during this survey and its behaviour recorded. The 33 unexercised endpoints are all writes; they need a disposable environment, not more specification.

12 Appendices

A Enumerations

ENUMERATION	VALUES
Order status	`Pending` , `Confirmed` , `Packed` , `Shipped` , `Delivered` , `Cancelled`
Payment status	`Unpaid` , `Paid`
Payment method	`COD` only in this build
Prescription status	`Pending` , `Approved` , `Rejected` ; `all` as a filter
User role	`Customer` , `Admin`
Catalogue sort	omitted, `relevance` , `price-asc` , `price-desc` , `name`
Inventory view	`low` , `out` , `nophoto` , `offers` , `featured` , `hidden` , `all`
Report key	`sales` , `orders` , `products` , `customers` , `prescriptions` , `payments`
Category key	`tablets-capsules` , `syrups-liquids` , `injections` , `skin-topicals` , `eye-ear-nose` , `respiratory` , `devices` , `baby-mother` , `personal-care` , `food-nutrition` , `essentials`
Department key	`medicines` , `health-wellness` , `personal-care` , `baby-care` , `skin-care` , `nutrition`
Prescription uploads	`.jpg .jpeg .png .webp .pdf` , maximum 8 MB
Image uploads	PNG, JPG, WEBP, SVG, maximum 3 MB

B Complete endpoint index

METHOD	PATH	AUTH	SECTION
GET	`/api/store/config`	none	4.1
GET	`/api/store/content`	none	4.1
GET	`/api/catalog/home`	none	4.2
GET	`/api/catalog/categories`	none	4.2
GET	`/api/catalog/manufacturers`	none	4.2
GET	`/api/catalog/districts`	none	4.2
GET	`/api/catalog/products`	none	4.2
GET	`/api/catalog/products/{id}`	none	4.2
GET	`/api/catalog/suggest`	none	4.2
GET	`/api/auth/me`	none	4.3
POST	`/api/auth/login`	none	4.3
POST	`/api/auth/register`	none	4.3
POST	`/api/auth/logout`	customer	4.3

METHOD	PATH	AUTH	SECTION
GET	`/api/auth/forgot/channels`	none	4.3
POST	`/api/auth/forgot/send`	none	4.3
POST	`/api/auth/forgot/reset`	none	4.3
GET	`/api/cart`	customer	4.4
PUT	`/api/cart/items/{productId}`	customer	4.4
DELETE	`/api/cart/items/{productId}`	customer	4.4
DELETE	`/api/cart`	customer	4.4
POST	`/api/cart/merge`	customer	4.4
POST	`/api/cart/quote`	none	4.4
GET	`/api/wishlist`	customer	4.5
GET	`/api/wishlist/ids`	customer	4.5
POST	`/api/wishlist/{productId}/toggle`	customer	4.5
GET	`/api/account/summary`	customer	4.6
PUT	`/api/account/profile`	customer	4.6
POST	`/api/account/password`	customer	4.6
GET	`/api/account/addresses`	customer	4.6
POST	`/api/account/addresses`	customer	4.6
POST	`/api/account/addresses/{id}/default`	customer	4.6
DELETE	`/api/account/addresses/{id}`	customer	4.6
GET	`/api/prescriptions`	customer	4.7
POST	`/api/prescriptions`	customer	4.7
GET	`/api/prescriptions/{id}/file`	customer / admin	4.7
GET	`/api/checkout/options`	customer	4.8
POST	`/api/checkout`	customer	4.8
GET	`/api/orders`	customer	4.8
GET	`/api/orders/{id}`	customer	4.8
POST	`/api/orders/{id}/cancel`	customer	4.8
POST	`/api/orders/{id}/reorder`	customer	4.8
POST	`/api/newsletter`	none	4.9
GET	`/api/admin/summary`	admin	5.1
GET	`/api/admin/dashboard`	admin	5.1

METHOD	PATH	AUTH	SECTION
GET	`/api/admin/orders`	admin	5.2
GET	`/api/admin/orders/{id}`	admin	5.2
POST	`/api/admin/orders/{id}/status`	admin	5.2
POST	`/api/admin/orders/{id}/payment`	admin	5.2
GET	`/api/admin/orders/{id}/outlets`	admin	5.2
GET	`/api/admin/prescriptions`	admin	5.3
POST	`/api/admin/prescriptions/{id}/review`	admin	5.3
DELETE	`/api/admin/prescriptions/{id}`	admin	5.3
GET	`/api/admin/inventory`	admin	5.4
GET	`/api/admin/products/{id}/content`	admin	5.4
PUT	`/api/admin/products/{id}/content`	admin	5.4
POST	`/api/admin/products/{id}/image`	admin	5.4
DELETE	`/api/admin/products/{id}/image`	admin	5.4
GET	`/api/admin/customers`	admin	5.5
GET	`/api/admin/manufacturers`	admin	5.6
POST	`/api/admin/manufacturers/{id}/logo`	admin	5.6
DELETE	`/api/admin/manufacturers/{id}/logo`	admin	5.6
GET	`/api/admin/content`	admin	5.7
PUT	`/api/admin/content/home`	admin	5.7
PUT	`/api/admin/content/pages`	admin	5.7
PUT	`/api/admin/content/faqs`	admin	5.7
POST	`/api/admin/content/images`	admin	5.7
GET	`/api/admin/settings`	admin	5.8
PUT	`/api/admin/settings/details`	admin	5.8
PUT	`/api/admin/settings/delivery`	admin	5.8
PUT	`/api/admin/settings/payments`	admin	5.8
PUT	`/api/admin/settings/social`	admin	5.8
GET	`/api/admin/reports/{key}`	admin	5.9
GET	`/api/admin/reports/{key}/export`	admin	5.9

C Reproducing the survey

Every claim in this document can be re-checked with curl. The guard header is the only thing that is not obvious:

```
# sign in and keep the cookie
curl -sk -X POST https://202.4.101.118:1165/api/auth/login \
  -H 'Content-Type: application/json' \
  -H 'X-Requested-With: NorthPharmacy' \
  -d '{"phone":"01784066929","password":"123Emsl!@#","rememberMe":true}' \
  -c cookies.txt

# an authenticated read
curl -sk -b cookies.txt -H 'X-Requested-With: NorthPharmacy' \
  https://202.4.101.118:1165/api/account/summary

# the guard itself — omit the header and watch it fail
curl -sk -X POST https://202.4.101.118:1165/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"phone":"01784066929","password":"123Emsl!@#"}'
# {"title":"Request rejected.","status":400}
```

D Glossary

TERM	MEANING IN THIS DOCUMENT
Guard header	`X-Requested-With: NorthPharmacy`, required on every request — §3.3
Problem document	An RFC 7807 error body — §3.4
Envelope	The `items` / `total` / `page` wrapper on collections — §3.6
Clamp	Silently adjusting an out-of-range parameter instead of rejecting it
Destructive	A case that writes data the whole team shares; gated behind `RUN_DESTRUCTIVE=1`
Blocked	A case that cannot run because the feature is switched off
Exercised	Actually called against the live environment during this survey
Rx	Prescription

E Revision history

VER	DATE	AUTHOR	CHANGE
1.0	25 Oct 2026	QA Automation	First issue. 73 endpoints documented from a live survey; 233 test cases specified; 8 findings raised

Keeping this document true

It describes a system that has no published contract, so it will drift the moment the API changes. Two habits keep it honest: re-run the survey in appendix C whenever the API version changes, and treat a contract-case failure in module API-CNV or API-CAT as a documentation bug as well as a code bug — fix both or neither.